

TECHNICAL HANDBOOK · 版本 0.9.4 · 2026-08-26

PNG 图像格式、解码算法与工程实现

从文件格式、Chunk、zlib、DEFLATE，到 Scanline 重建、Adam7、颜色处理与硬件架构

00. 前言

从文件格式、Chunk、zlib、DEFLATE，到 Scanline 重建、Adam7、颜色处理与硬件架构

版本：0.9.4

日期：2026-08-26

适用对象：PNG 初学者、软件解码器开发者、FPGA/ASIC/RTL 设计工程师

1. 前言

PNG (Portable Network Graphics) 不是“把像素直接交给 DEFLATE 压缩”这么简单。它是一套分层的数据格式：

1. 最外层是 PNG Signature 和一系列 Chunk；
2. IDAT Chunk 的 Data 字段拼接成一个 zlib 数据流；
3. zlib 内部包裹一个 DEFLATE 位流，并在尾部提供 Adler-32；
4. DEFLATE 解压得到的不是最终像素，而是带有行过滤器类型的 Filtered Scanline；
5. Decoder 必须执行 Unfilter、Sample Unpack、Palette 与 Transparency Processing；若使用 Adam7，还要执行 Adam7 Placement，将七个 Pass 的像素放回原图位置。

本书以解码器为主线，同时补充编码器和硬件实现。核心数据路径如下。

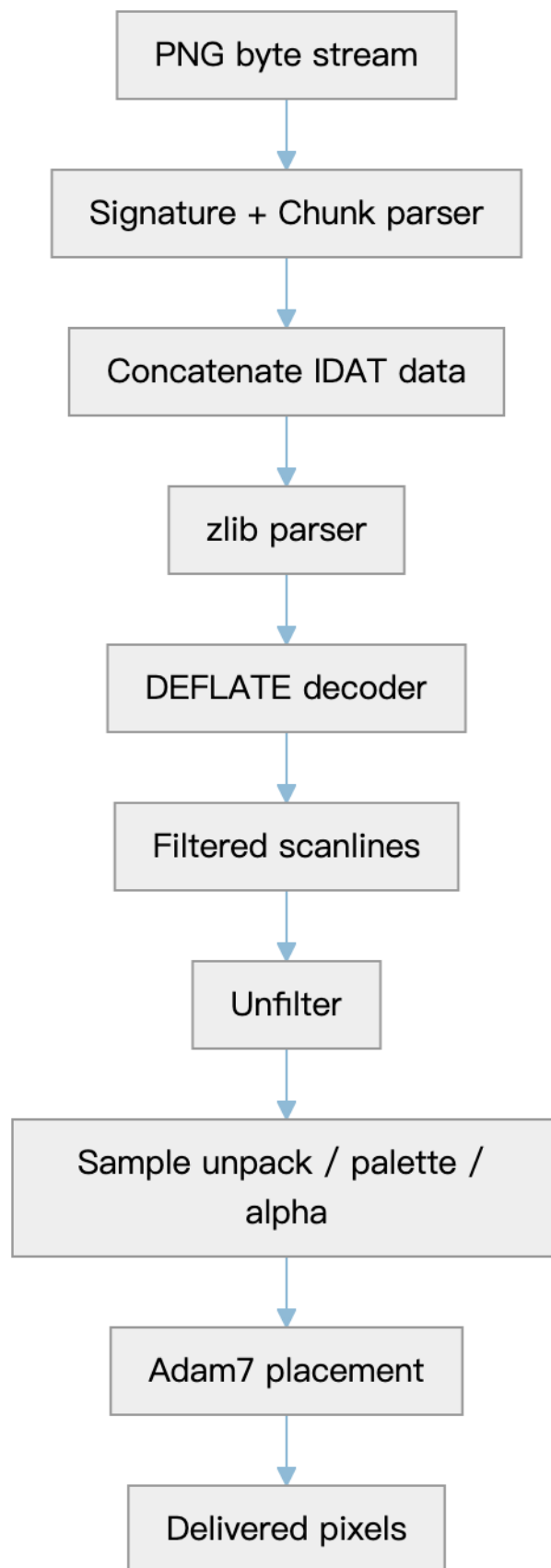


图 1

1.1 规范依据

本书主要依据以下标准：

- W3C, Portable Network Graphics (PNG) Specification, Third Edition, W3C Recommendation, 24 June 2025;
- ISO/IEC 15948:2004, 与 PNG Second Edition 对应;
- RFC 1950, ZLIB Compressed Data Format Specification version 3.3;
- RFC 1951, DEFLATE Compressed Data Format Specification version 1.3;
- W3C PNG Fourth Edition Editor's Draft, 用于了解后续演进，不把草案内容误写成既有实现的强制要求。

规范链接见文末参考资料。

1.2 阅读约定

- byte 固定指 8 bit;
- PNG 多字节整数采用网络字节序，即最高有效字节在前;
- 所有 Scanline Filter 运算都按 byte 进行，并对 256 取模;
- 伪代码强调算法含义，不绑定具体语言;
- 本文中的“必须”“不得”表示格式或一致性约束；“建议”表示工程选择。

1.3 目录

- 第一篇 建立整体认识
- 第二篇 PNG 文件格式与 Chunk
- 第三篇 像素与 Scanline
- 第四篇 完整 Decoder 数据流
- 第五篇 zlib
- 第六篇 DEFLATE 解码
- 第七篇 Scanline Filter 与重建
- 第八篇 Adam7 原理与解码流程
- 第九篇 颜色、Alpha 与输出
- 第十篇 编码器
- 第十一篇 硬件架构
- 第十二篇 错误、安全与验证
- 第十三篇 贯通实例
- 第十四篇 PNG 图像测试集与验证资源汇总
- 附录 A~E 速查公式、常见误解、APNG 概览、规范版本关系与参考资料

01. 建立整体认识

1. PNG 是什么

PNG 是一种无损、可扩展、可流式解析的栅格图像格式。它支持：

- 灰度、真彩色和索引色；
- 1、2、4、8、16 bit Sample Depth；
- 可选 Alpha；
- Gamma、色度、ICC、HDR 标识等颜色信息；
- 文本、Exif、物理尺寸、时间等元数据；
- Adam7 渐进显示；
- CRC 和 Adler-32 两层完整性检查；
- 在较新规范中定义的 APNG 帧动画。

“无损”意味着：忽略显示设备和颜色管理造成的视觉转换，Decoder 能逐 Sample 恢复编码器提交给 PNG 编码过程的数据。

PNG 的压缩效果来自两层协作：

1. Scanline Filter 把空间相关性转化为大量小残差；
2. DEFLATE 用 LZ77 查找重复串，再用 Huffman Code 表示 Literal、Length 和 Distance。

Filter 自身不减少字节数。每行反而多出一个 Filter Type Byte；真正减少数据量的是后面的 DEFLATE。

2. 编码与解码的对称关系

编码路径：

```
Reference pixels
-> optional Adam7 pass extraction
-> scanline serialization
-> per-scanline filtering
-> concatenate all filtered scanlines
-> zlib/DEFLATE compression
-> split compressed bytes across IDAT chunks
-> construct PNG chunks and CRCs
```

解码路径：

```
PNG bytes
-> parse chunks and verify CRC
-> concatenate IDAT data fields
-> parse zlib header
-> decode DEFLATE
-> verify Adler-32
-> split output into scanlines for the image or each Adam7 pass
-> reverse the scanline filters
-> unpack samples and apply palette/transparency
-> place Adam7 samples into final coordinates
```

需要牢牢记住三个边界：

- IDAT Chunk 边界只是 PNG 封装边界；
- DEFLATE Block 边界是压缩算法边界；
- Scanline 边界是图像预测边界。

三者通常互不对齐。Decoder 不能假设“一个 IDAT 对应一个 DEFLATE Block”或“一个 Block 对应一行”。

3. 六层数据模型

层次	输入	输出	主要职责
PNG 文件层	文件字节	Chunk	Signature、顺序、长度、CRC
IDAT 聚合层	一个或多个 IDAT	连续 zlib 字节流	去掉 Chunk 外壳并按顺序拼接 Data
zlib 层	zlib 字节流	DEFLATE 位流与校验结果	CMF/FLG、Adler-32
DEFLATE 层	压缩位流	Filtered Scanline 字节	Block、Huffman、LZ77
Filter 层	Filter Type + Filtered Bytes	原始 Scanline Bytes	None/Sub/Up/Average/Paeth
像素层	Scanline Bytes	像素或 Sample	位深解包、PLTE、tRNS、Adam7

这种分层也应直接反映在软件模块或 RTL 模块划分中。

02. PNG 文件格式与 Chunk

PNG 压缩层级结构

1. PNG Signature

每个 PNG 数据流以前8字节开始:

HEX: 89 50 4E 47 0D 0A 1A 0A

其中 50 4E 47 是 ASCII 的“PNG”。其余字节帮助检测:

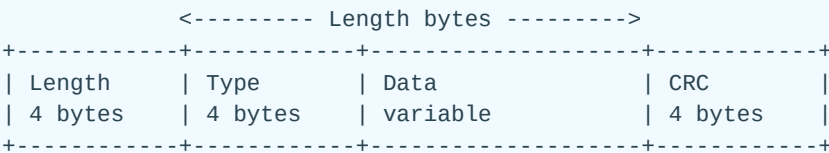
- 文本模式与二进制模式混淆;
- CR/LF 换行转换;
- 文件截断;
- 某些旧式传输链路对控制字符的破坏。

Decoder 应在分配大块图像内存之前检查完整 Signature。

2. Chunk 通用结构

每个 Chunk 均为:

字段	大小	含义
Length	4 byte	Data 字段长度, 不含 Type 和 CRC
Type	4 byte	四个 ASCII 字母
Data	Length byte	Chunk-specific payload
CRC	4 byte	对 Type 与 Data 计算的 CRC-32



Length、宽高及其他 PNG 整数均按大端字节序读取。实现时必须先做边界与溢出检查, 再计算:

chunk_total = 12 + length

不要在未验证 length 的情况下直接执行内存分配或指针加法。

3. Chunk Type 的四个属性位

Chunk Type 的四个字符必须是英文字母。每个字符 ASCII 值的 bit 5，即大小写差异，具有独立含义。

字符位置	大写, bit 5=0	小写, bit 5=1
第1字符	Critical	Ancillary
第2字符	Public	Private
第3字符	Reserved, 当前定义的类型使用大写	Reserved bit=1, 数据流不符合当前版本
第4字符	Unsafe-to-copy	Safe-to-copy

例如 IDAT 的首字符 I 为大写，所以它是 Critical Chunk。tEXt 首字符小写，所以它是 Ancillary Chunk。

Decoder 不应仅因未知 Chunk 的第三字符为小写就立即报错，而应把它当作未知 Chunk，再根据第一字符的 Critical/Ancillary 属性处理。这样既能指出数据流不符合当前版本，也能保持对未来 Ancillary Chunk 的兼容性。

未知 Critical Chunk 意味着 Decoder 不知道如何正确解释图像，必须报错。未知 Ancillary Chunk 通常可以跳过，但编辑器是否可在修改图像后原样保留，还要看第四字符的 safe-to-copy 属性。

常见 PNG 与 APNG Chunk Type 如下：

类别	Chunk Type	意义说明
Critical	IHDR	图像头；定义宽高、位深、颜色类型、压缩、Filter 和交错方法
Critical	PLTE	调色板；为索引色提供 RGB 条目，也可为真彩色提供建议调色板
Critical	IDAT	图像数据；所有连续 Data 按顺序拼接为一个 zlib 数据流
Critical	IEND	图像结束标记；Data 长度必须为 0

Ancillary Chunk 详见第 8 节。

4. IHDR：图像的总合同

IHDR 必须是 Signature 后的第一个 Chunk，只出现一次，Data 长度固定13字节。

偏移	字段	大小
0	Width	4
4	Height	4
8	Bit Depth	1
9	Color Type	1
10	Compression Method	1
11	Filter Method	1
12	Interlace Method	1

Width 和 Height 必须非零，且不能超过 PNG 四字节无符号整数允许的范围。

4.1 Color Type 与通道数

Color Type	含义	通道顺序	通道数
0	Grayscale	G	1
2	Truecolor	R, G, B	3
3	Indexed-color	Palette Index	1
4	Grayscale with Alpha	G, A	2
6	Truecolor with Alpha	R, G, B, A	4

Color Type 的 bit 含义可以帮助记忆，但实现必须只接受规范定义的 0、2、3、4、6。

4.2 合法 Bit Depth

Color Type	1	2	4	8	16
Grayscale	✓	✓	✓	✓	✓
Truecolor				✓	✓
Indexed-color	✓	✓	✓	✓	
Grayscale with Alpha				✓	✓
Truecolor with Alpha				✓	✓

4.3 三个 Method 字段

- Compression Method：当前只定义0；
- Filter Method：当前只定义0；

- Interlace Method: 0 表示无交错, 1 表示 Adam7。

PNG Compression Method 0 规定使用 zlib/DEFLATE。这里的“0”是 PNG 的方法编号, 不是 zlib Header 中的 CM 值。

5. PLTE: 调色板

PLTE Data 由若干个三字节条目组成:

```
R0 G0 B0 | R1 G1 B1 | ...
```

约束包括:

- 长度必须是3的倍数;
- 条目数为1至256;
- Color Type 3 必须有 PLTE;
- Color Type 0 和4 不得出现 PLTE;
- Color Type 2 和6 可以提供建议调色板;
- PLTE 必须位于第一个 IDAT 之前;
- Color Type 3 的条目数不得超过位深能表示的索引数量;
- 解码得到的索引超出实际 PLTE 条目数属于错误。

PLTE 不进入 DEFLATE。它是独立 Chunk。

6. IDAT: 图像压缩数据

IDAT 的 Data 字段包含 zlib 数据流的一部分。PNG 可以有一个或多个 IDAT, Decoder 必须把所有连续 IDAT 的 Data 按文件顺序逻辑拼接:

```
IDAT_0.Data || IDAT_1.Data || ... || IDAT_n.Data
```

拼接结果是一个 zlib datastream:

```
CMF | FLG | DEFLATE blocks ... | ADLER32
```

关键结论:

1. IDAT 的 Length、Type 和 CRC 不属于 zlib;
2. IDAT Data 开头不一定是新的 zlib Header, 只有整个拼接流开头才是;
3. Chunk 可以在任意压缩字节位置切分;
4. Chunk 边界甚至可能落在一个 Huffman Code 所在字节附近;

5. Decoder 可以流式地把每个 IDAT Data 送入同一个持久化 zlib 状态机，不必先复制到一块大缓冲。

严格说，“去掉 IDAT Header 后 body 就是 zlib 标准格式”只对所有 IDAT Data 的有序拼接成立，不对每个 IDAT 单独成立。

7. IEND 与 Critical Chunk 顺序

IEND:

- Data 长度必须为0;
- 只出现一次;
- 必须是最后一个 PNG Chunk。

静态 PNG 的关键顺序可概括为:

IHDR
[PLTE]
IDAT ...
IEND

所有 IDAT 必须连续出现。如果已经离开 IDAT 序列又遇到 IDAT，应视为顺序错误。

8. Ancillary Chunk 速览

类别	Chunk	作用
透明度	tRNS	透明色或 Palette Alpha
颜色	cHRM	白点和原色色度
颜色	gAMA	图像 Gamma
颜色	iCCP	内嵌 ICC Profile
颜色	sRGB	标准 sRGB 意图
颜色	cICP	Color Primaries、Transfer、Matrix、Range
HDR	mDCv	Mastering Display Color Volume
HDR	cLLi	Content Light Level
精度	sBIT	原始有效位数
文本	tEXt	Latin-1 文本
文本	zTXt	压缩 Latin-1 文本
文本	iTXt	UTF-8 国际文本
显示	bKGD	建议背景色

类别	Chunk	作用
调色板	hIST	Palette 频率
物理	pHYs	像素密度或宽高比
调色板	sPLT	建议调色板
元数据	eXIf	Exif Profile
时间	tIME	最后修改时间
动画	acTL/fcTL/fdAT	APNG 控制、帧控制和帧数据

基础图像 Decoder 可以忽略多数不理解的 Ancillary Chunk，但不能忽略影响核心像素解释的 IHDR、PLTE、IDAT、IEND，也不能在遇到未知 Critical Chunk 后继续声称解码正确。

8.1 tRNS

tRNS 的结构依赖 Color Type:

- Type 0: 一个灰度 Sample 值;
- Type 2: R、G、B 三个 Sample 值;
- Type 3: 按 Palette Index 排列的 Alpha byte，未提供的尾部条目默认为255;
- Type 4、6 已有 Alpha Channel，不允许 tRNS。

对 Type 0/2，透明判断必须在原始 Sample 精度下做完全相等比较。

8.2 颜色 Chunk

颜色管理不是恢复像素编码值的必要条件，却是正确显示颜色的重要条件。工程上至少应区分：

- “Decoder 恢复了 PNG Sample”;
- “Viewer 把 Sample 转换为显示设备颜色”。

Alpha 总是线性的，不能套用颜色通道的 Gamma 曲线。PNG 存储的是非预乘 Alpha（straight/unassociated alpha）；如果输出接口要求 premultiplied alpha，应在颜色空间处理正确后显式转换。

9. CRC-32

每个 Chunk 的 CRC 覆盖：

Type bytes Data bytes

不覆盖 Length。PNG 使用标准 CRC-32 多项式，反射实现常用 0xEDB88320。典型流程：

```
crc = 0xFFFFFFFF
for byte in Type || Data:
    crc = update_crc(crc, byte)
crc = crc XOR 0xFFFFFFFF
```

流式 Parser 可以在读取 Type 和 Data 时同步更新 CRC，无需缓存整个 Chunk。

CRC 的价值是定位 Chunk 级传输错误；zlib 尾部 Adler-32 则覆盖解压后的全部 Filtered Scanline 字节。两者保护的对象和层级不同。

02-1 Grayscale 转 RGB

PNG 的 Grayscale（灰度，Color Type = 0）像素只有一个灰度分量。显示为 RGB 时，三个颜色通道取相同值：

$$R = G = B$$

关键在于：根据 PNG 的 bit depth，把原始灰度 sample 映射到目标 RGB 位深。

1. Grayscale 支持的 Bit Depth

对于 PNG Color Type = 0（Grayscale），合法的 bit depth 为：

Bit Depth	原始灰度范围	灰度级数
1	0 ~ 1	2
2	0 ~ 3	4
4	0 ~ 15	16
8	0 ~ 255	256
16	0 ~ 65535	65536

其中：

- 0 表示黑色；
- 最大值表示白色；
- 中间值按比例表示不同灰度。

2. 转换成 8-bit RGB 的通用公式

假设 PNG 的 bit depth 为 b ，原始灰度 sample 为：

$$g_{raw} \in [0, 2^b - 1]$$

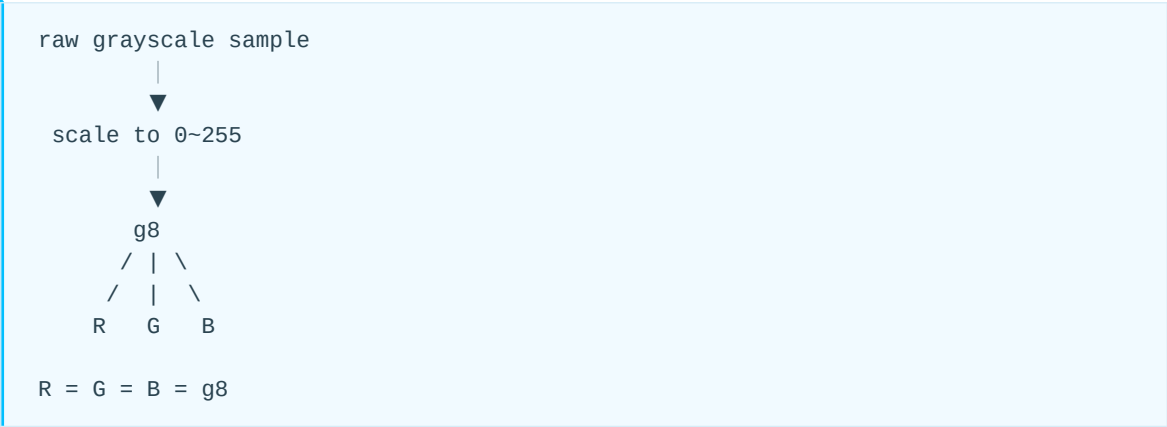
转换成标准 8-bit 灰度值：

$$g_8 = \text{round} \left(g_{raw} \times \frac{255}{2^b - 1} \right)$$

然后：

$$R = G = B = g_8$$

流程可以表示为：



3. Bit Depth = 1

1-bit grayscale 只有两个灰度值：

0 → black
1 → white

对应关系：

Raw	RGB
0	(0, 0, 0)
1	(255, 255, 255)

因为：

$$g_8 = g_1 \times 255$$

例如一个 scanline 中的数据：

0 1 1 0 1 0 0 1

对应：

black white white black white black black white

RGB 为：

(0, 0, 0)
(255, 255, 255)
(255, 255, 255)
(0, 0, 0)
...

4-bit	8-bit	RGB
0x0	0	(0,0,0)
0x1	17	(17,17,17)
0x2	34	(34,34,34)
0x3	51	(51,51,51)
0x4	68	(68,68,68)
...	...	...
0x8	136	(136,136,136)
...	...	...
0xF	255	(255,255,255)

这也可以理解为 bit replication:

```
4 bit:
1010
扩展为 8 bit:
1010 1010
```

即:

```
0xA → 0xAA = 170
```

等价于:

$$g_8 = (g_4 << 4) | g_4$$

6. Bit Depth = 8

8-bit grayscale 已经处于 0~255 范围，因此无需缩放:

$$g_8 = g_{raw}$$

例如:

```
Gray = 0x00  
→ RGB = (0, 0, 0)  
  
Gray = 0x40  
→ RGB = (64, 64, 64)  
  
Gray = 0x80  
→ RGB = (128, 128, 128)  
  
Gray = 0xFF  
→ RGB = (255, 255, 255)
```

代码形式:

CPP

```
uint8_t gray = sample;  
  
R = gray;  
G = gray;  
B = gray;
```

7. Bit Depth = 16

16-bit grayscale sample 范围为:

0 ~ 65535

如果输出仍然使用 16-bit RGB, 则直接:

$$R_{16} = G_{16} = B_{16} = g_{16}$$

例如:

```
Gray = 0x1234  
  
R = 0x1234  
G = 0x1234  
B = 0x1234
```

如果要转换成 8-bit RGB:

$$g_8 = \text{round} \left(g_{16} \times \frac{255}{65535} \right)$$

由于:

$$65535 = 255 \times 257$$

所以可以写成:

$$g_8 \approx \frac{g_{16}}{257}$$

例如：

Gray16	Decimal	Gray8
0x0000	0	0
0x4040	16448	64
0x8080	32896	128
0xFFFF	65535	255

特别是：

```
0x8080 → 0x80
0xFFFF → 0xFF
```

这可以理解成一个 8-bit 值扩展到 16-bit：

```
8-bit:
10000000

16-bit:
10000000 10000000
```

8. 1/2/4 bit 并不是“一个像素占一个 byte”

对于低 bit-depth grayscale，多个像素会打包在同一个 byte 中。

Bit Depth = 1

一个 byte 保存 8 个像素：

```
10110010
| | | | |
8 grayscale pixels
```

Bit Depth = 2

一个 byte 保存 4 个像素：

```
10 01 11 00
|  |  |  |
4 pixels
```

Bit Depth = 4

一个 byte 保存 2 个像素：

```
1010 0011
|      |
pixel pixel
```

Bit Depth = 8

一个 byte 保存 1 个像素：

```
xxxxxxxx
|
1 pixel
```

Bit Depth = 16

两个 byte 保存 1 个像素：

```
xxxxxxxx xxxxxxxx
|
1 pixel
```

PNG 的 16-bit sample 使用 network byte order / big-endian：

```
0x1234

文件中：

12 34
↑  ↑
MSB LSB
```

9. Grayscale 解码到 RGB 的完整关系

对于 PNG Analyzer，可以将 Raw Sample 和 Displayed RGB 明确区分：



统一公式为：

$$RGB = \left(S \frac{255}{2^b - 1}, S \frac{255}{2^b - 1}, S \frac{255}{2^b - 1} \right)$$

其中：

- S: 原始 grayscale sample;
- b: PNG bit depth;
- 最终三个 RGB 通道取相同值。

10. 8-bit RGB 速查表

PNG Bit Depth	原始范围	转换到 8-bit
1	0~1	gray × 255
2	0~3	gray × 85
4	0~15	gray × 17
8	0~255	gray
16	0~65535	gray / 257 (近似表达)

最终均为：

$$R = G = B$$

11. Grayscale 与 tRNS

对于 Color Type = 0，PNG 仍然可以包含 tRNS chunk。

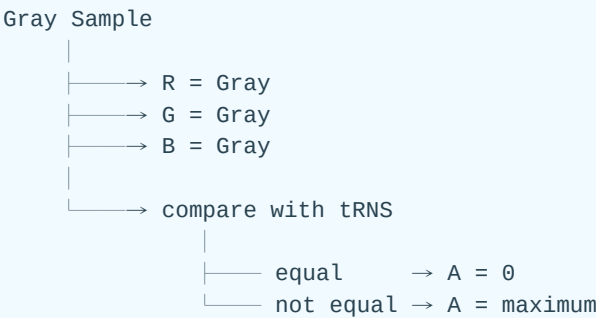
tRNS 可以指定一个特定的 grayscale sample 为完全透明。因此，如果最终输出 RGBA，不能简单地始终令：

A = 255

而应比较当前 grayscale sample 是否等于 tRNS 中定义的透明灰度值：

```
if gray_sample == transparent_gray:
    A = 0
else:
    A = max_alpha
```

因此，对于带 tRNS 的 grayscale PNG，最终可理解为：



03. 像素与 Scanline

1. Pixel、Sample、Channel

Pixel 是空间位置；Channel 是 G、R、B、A 或 Palette Index 等分量；Sample 是某个 Pixel 与某个 Channel 的交点。

例如 8-bit RGBA：

```
Pixel 0: R0 G0 B0 A0
Pixel 1: R1 G1 B1 A1
```

Bits per pixel：

```
bits_per_pixel = channels * bit_depth
```

一条非交错 Scanline 的数据字节数：

```
row_bytes = ceil(width * bits_per_pixel / 8)
            = (width * bits_per_pixel + 7) / 8
```

实现必须用足够宽的整数并检查乘法溢出。

2. Sample 打包

2.1 Bit Depth 小于8

1、2、4-bit Sample 从每个 byte 的最高有效位向最低有效位打包。每行独立开始，末尾不足一个 byte 的低位是 Padding，不属于图像 Sample。

例如 2-bit 灰度 Sample：

```
samples:  3, 1, 0, 2
bits:     11 01 00 10
byte:     11010010 = D2
```

不能把上一行末尾的剩余 bit 与下一行拼在一起。

2.2 Bit Depth 16

每个 16-bit Sample 按大端顺序：

```
sample = high_byte << 8 | low_byte
```

RGBA16 的一个像素占8 byte，顺序为 R高、R低、G高、G低、B高、B低、A高、A低。

3. Filtered Scanline 格式

DEFLATE 解压后，每个非空 Scanline 的格式是：

```
FilterType | filtered_byte[0] | ... | filtered_byte[row_bytes-1]
```

因此非交错图像的预期解压长度为：

```
expected = height * (1 + row_bytes)
```

Adam7 图像要分别计算七个 Pass，每个非空 Pass 的每行也多一个 Filter Type Byte。

3.1 Filter 算法中的 bpp

Filter 公式中的 bpp 定义为一个完整像素占用的字节数，向上取整：

```
bpp = max(1, ceil(bits_per_pixel / 8))
```

例如：

格式	bits_per_pixel	Filter bpp
Grayscale 1-bit	1	1
Indexed 4-bit	4	1
RGB8	24	3
RGBA8	32	4
RGB16	48	6

Filter 操作对象是序列化后的 byte，不是抽象的 Sample 或 Pixel。低位深格式中，一个 byte 包含多个像素，但 Sub 的左邻居仍是前一个 byte。

04. 完整 Decoder 数据流

1. Parser 状态机

推荐的顶层状态机：

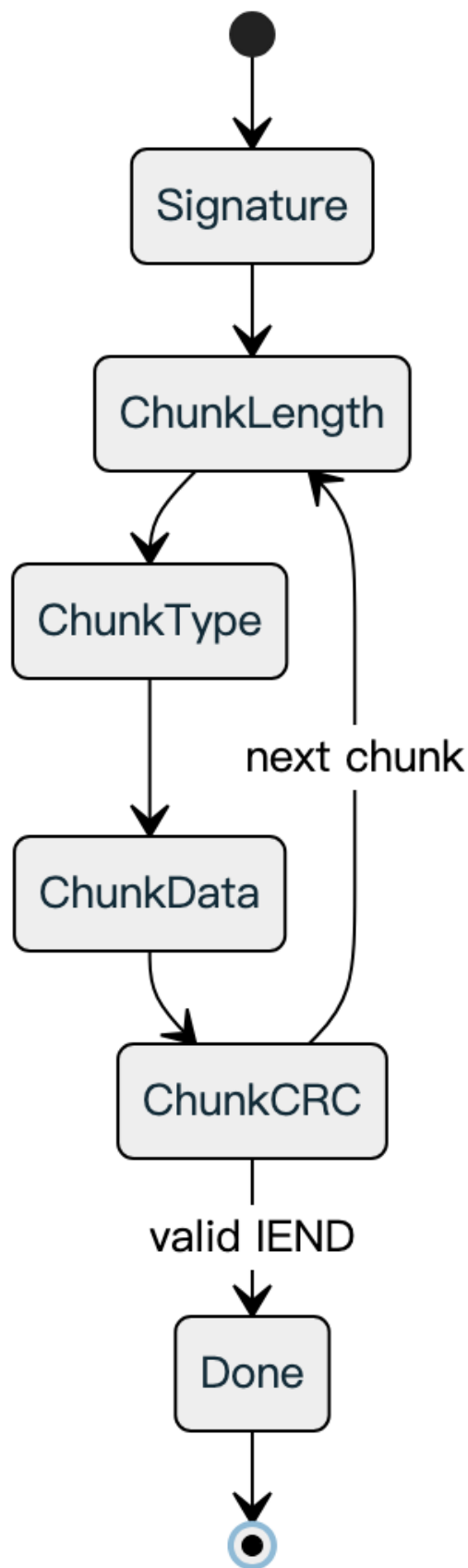


图 2

Parser 应维护：

- 是否已见 IHDR、PLTE、IDAT、IEND；
- 当前是否仍位于连续 IDAT 区间；
- Color Type 与后续 Chunk 的兼容性；
- 当前 Chunk 剩余字节；
- CRC 状态；
- 整体错误状态。

对 IDAT，Chunk Parser 只剥离外壳，把 Data byte 送给同一 zlib 输入接口。

2. 流式解码与缓冲

完整 PNG 解码并不要求：

- 缓存整个文件；
- 缓存所有 IDAT；
- 缓存完整解压结果；
- 缓存完整图像。

基础流式 Decoder 所需的主要存储：

存储	典型大小	用途
Bit Buffer	数十 bit	DEFLATE 位解析
Huffman Table	数百至数千项	符号解码
LZ77 Window	最大32 KiB	回溯复制
Current Row	row_bytes	当前反滤波
Previous Row	row_bytes	Up/Average/Paeth
Palette	最大256×RGB(A)	Indexed-color

若输出端必须按 Tile 或 Block 接收，则需要额外的行到块重排缓存。这是输出格式需求，不是 PNG 语法本身要求。

3. 边界互不对齐

一个稳健实现应把数据看成连续流，并在不同模块独立维护边界：

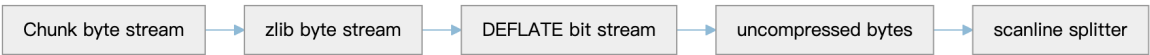


图 3

典型场景：

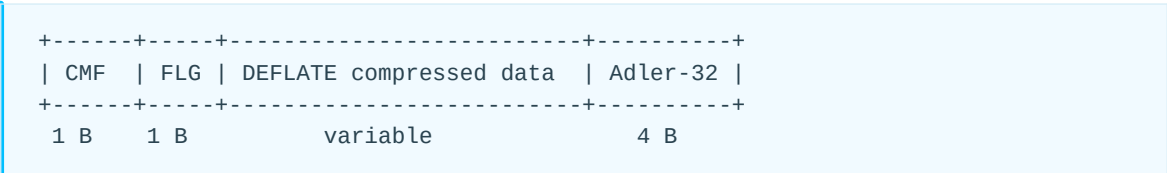
- zlib Header 的两个 byte 可以横跨内部总线传输拍；
- 一个 Dynamic Huffman Header 可以跨 IDAT；
- 一个 Length/Distance Copy 可以跨 Scanline；
- 一个 DEFLATE Block 可以覆盖多行；
- 一行也可能跨多个 DEFLATE Block。

因此解压器不理解行；行重建器也不应理解 DEFLATE Block。

05. zlib

1. zlib 与 DEFLATE 的关系

- zlib 是封装格式
- DEFLATE 是内部压缩编码



其中 DEFLATE compressed data 是 zlib 载荷：由一个或多个 DEFLATE 压缩块组成的位流（bit stream），解压后得到 PNG 的 Filter Type Byte 和 Filtered Scanline Bytes。

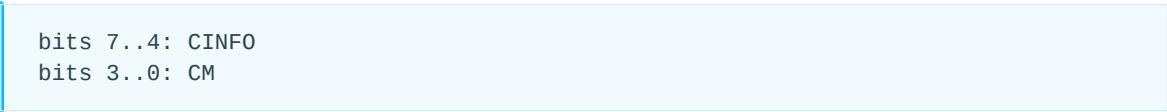
zlib Parser 的职责：

1. 解析和检查 CMF、FLG；
2. 检查 PNG 不允许的 preset dictionary；
3. 把中间位流送给 DEFLATE；
4. 对 DEFLATE 输出 byte 计算 Adler-32；
5. 与流尾校验值比较。

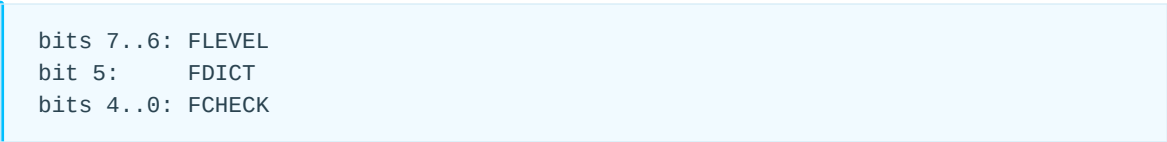
DEFLATE Decoder 不负责 PNG Chunk CRC，也不负责 Scanline Filter。

2. CMF 与 FLG

CMF:



FLG:



2.1 CM

PNG Compression Method 0 使用 zlib 的 CM=8，即 DEFLATE。虽然 RFC 1950 为未来压缩方法保留了其他 CM 值，PNG Decoder 不能因此接受任意 CM。

2.2 CINFO

对 CM=8:

```
window_size = 2^(CINFO + 8)
```

CINFO 最大为7，对应32768 byte。PNG 使用的窗口不得超过32 KiB；小于32 KiB 的窗口表示也是合法的通用情形，Decoder 的32 KiB Window 能覆盖它。

2.3 FCHECK

两个 Header byte 必须满足：

```
(CMF * 256 + FLG) mod 31 == 0
```

2.4 FDICT

PNG 不允许使用 preset dictionary，因此 FDICT 必须为0。若为1，通用 zlib 后面会有 DICTID，但 PNG Decoder 应直接报告非法数据流。

2.5 FLEVEL

FLEVEL 是编码器压缩等级提示，不改变解码算法。Decoder 不应根据它拒绝合法码流。

3. Adler-32

Adler-32 对所有 DEFLATE 解压输出 byte 计算，即覆盖 Filter Type Byte 和 Filtered Scanline Bytes。

初始化：

```
s1 = 1  
s2 = 0  
MOD = 65521
```

对每个输出 byte d:

```
s1 = (s1 + d) mod MOD  
s2 = (s2 + s1) mod MOD
```

最终：

```
adler = (s2 << 16) | s1
```

流中的 Adler-32 以最高有效字节在前存放。高性能实现可批量累加后再取模，但必须限制中间值，避免整数溢出。

CRC 与 Adler-32 的比较:

项目	PNG CRC-32	zlib Adler-32
覆盖对象	每个 Chunk 的 Type+Data	全部未压缩数据
位置	每个 Chunk 尾部	zlib 数据流尾部
主要层次	PNG 封装	zlib 内容
算法	多项式 CRC	两级模和

06. DEFLATE Decoder 算法 - RFC 1951

可以把 RFC 1951 的 DEFLATE Decoder 理解成两个核心动作的组合：

Huffman 解码：告诉你“这是一个 literal，还是一个 length/distance 描述”
LZ77 重建：如果是 length/distance，就从已经输出的数据中向后拷贝。

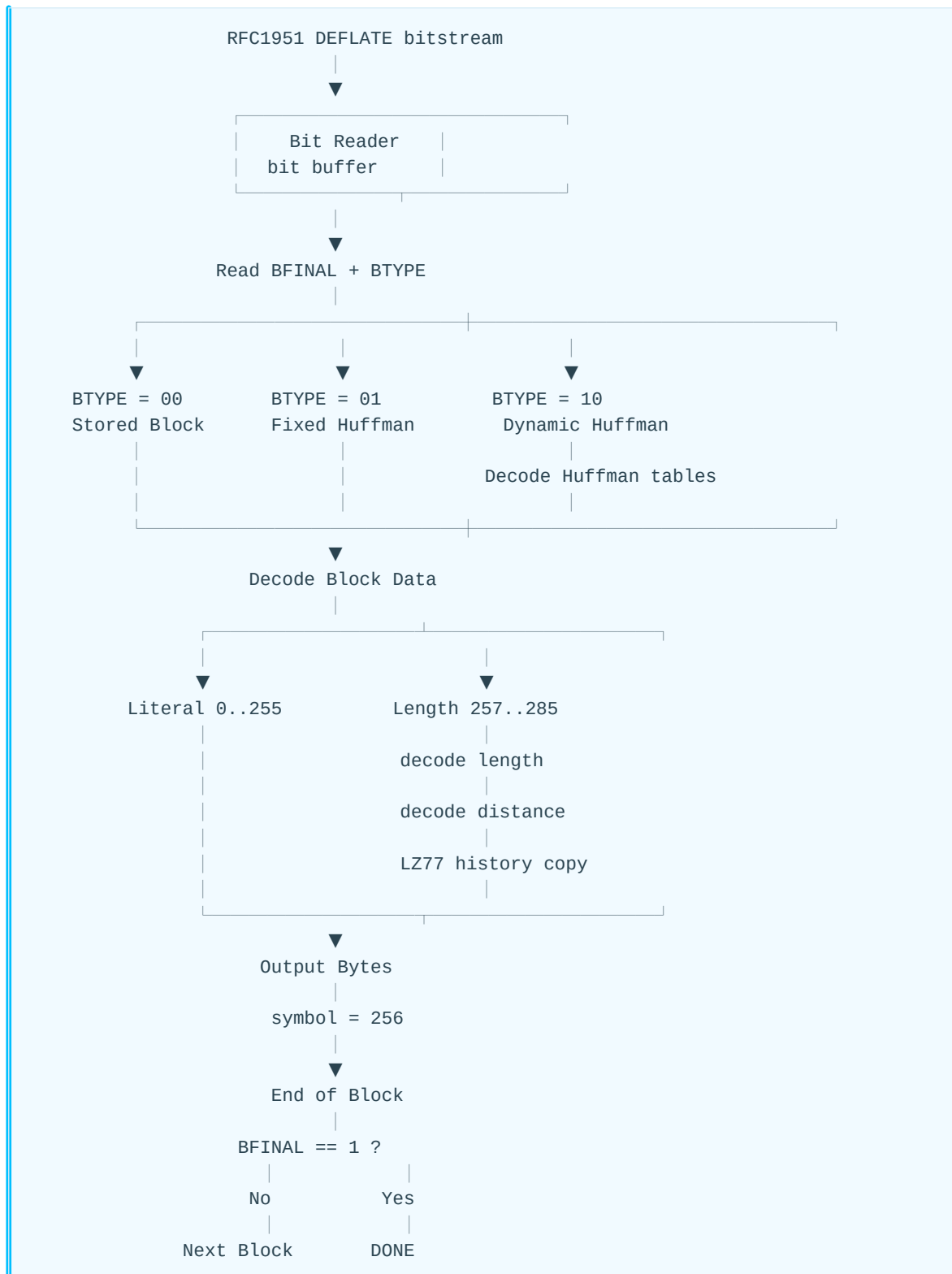
RFC 1951 定义的 DEFLATE 数据由一个或多个 block 顺序组成；每个 block 独立使用自己的 Huffman 表，但 LZ77 的历史窗口可以跨 block，最多向前引用 32 KiB。

参考规范：

- RFC 1951: DEFLATE Compressed Data Format Specification version 1.3
- <https://www.rfc-editor.org/rfc/rfc1951.html>

1. DEFLATE Decoder 的总体流程

先给一个最重要的全局图：

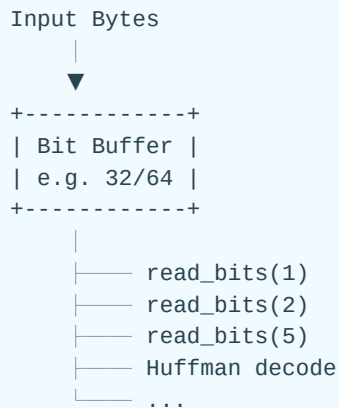


RFC 本身给出的 Decoder 核心伪代码基本就是这个结构。

2. Step 0: Bit Reader

DEFLATE 首先是一个 **bit-oriented syntax**，所以 decoder 前端通常不会直接按 byte 解析，而是维护一个 bit buffer。

一个典型硬件/软件结构是：



这里有 DEFLATE 最容易搞错的规则之一：

普通字段：LSB-first

RFC 1951 规定，一个 byte 内的数据元素从 bit 0，即 LSB 开始填充/读取；非 Huffman 数据元素自身也按最低有效 bit 开始打包。

例如：

```

byte = abcdefgh
      76543210

bitstream读取顺序：

h → g → f → e → d → c → b → a
  
```

所以后面看到：

```

BFINAL : 1 bit
BTYPE  : 2 bits
HLIT   : 5 bits
  
```

decoder 的 `read_bits(n)` 通常就是：

```

C
value = bitbuf & ((1 << n) - 1);
bitbuf >>= n;
  
```

3. Step 1：读取 Block Header

每个 DEFLATE block 开始有：

```
+-----+-----+
| BFINAL | BTYPE |
| 1 bit  | 2 bits |
+-----+-----+
```

RFC 定义：

BTYPE	类型
00	Stored / Uncompressed
01	Fixed Huffman
10	Dynamic Huffman
11	Reserved → Error

BFINAL = 1 表示：

这是整个 DEFLATE stream 的最后一个 block。

注意：

BFINAL != End-of-Block

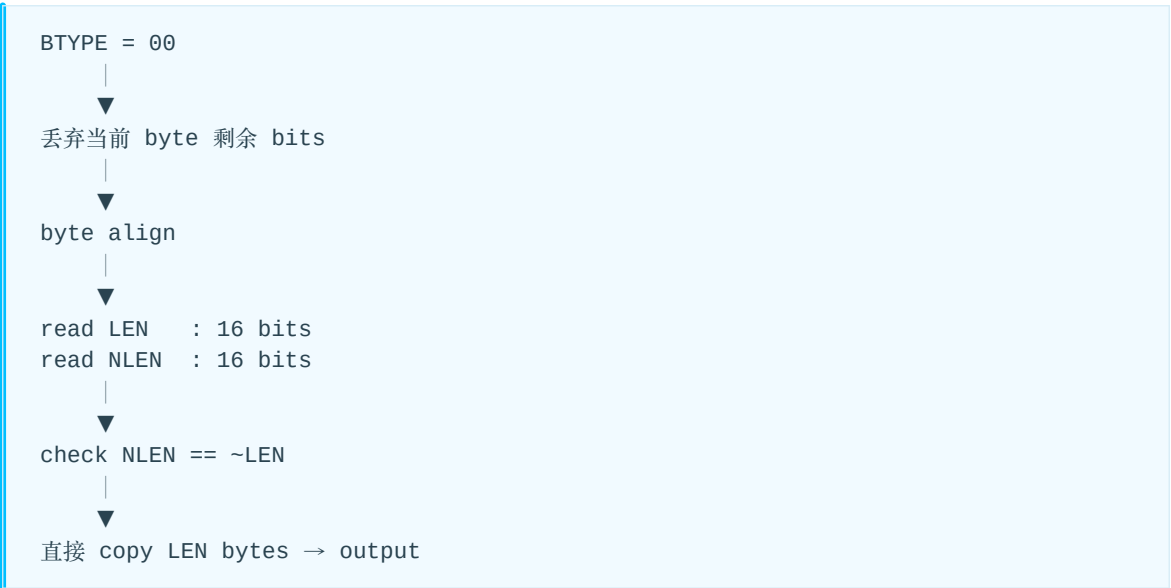
两者不是一回事。

- BFINAL：这个 block 是不是最后一个 block
- Huffman symbol 256：当前 compressed block 结束

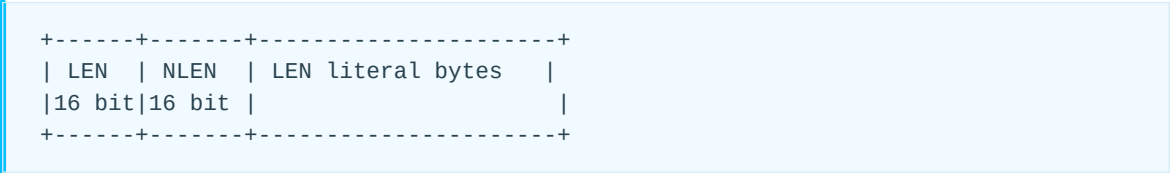
4. BTYPE = 00: Stored Block

这是最简单的一种。

流程：



格式:



这里 LEN ≤ 65535。

因此 Stored Block 完全没有:

- Huffman
- Length
- Distance
- LZ77 copy

就是直接输出原始 byte。

5. BTYP = 01: Fixed Huffman

从这里开始进入真正的 DEFLATE 核心。

Fixed Huffman 的特点是:

Huffman code table 不需要从 bitstream 传输，因为 RFC 已经固定规定。

Literal/Length alphabet 的 code length 为:

Symbol	Huffman bits
0 – 143	8
144 – 255	9
256 – 279	7
280 – 287	8

Distance code 0-31 固定是 5 bit code。

所以 Decoder 可以在初始化时就准备好：

Fixed Literal/Length Huffman Table

Fixed Distance Huffman Table

然后直接进入：

decode symbols

6. BTYPE = 10: Dynamic Huffman

这是 DEFLATE Decoder 中最复杂的部分。

它的本质是：

bitstream 先告诉 Decoder：本 block 使用什么 Huffman tree。

但它不是直接把整棵树传进来。

而是经过了一层压缩：



所以 Dynamic Huffman 有一种很有意思的“Huffman 解 Huffman”结构。

7. Dynamic Huffman Step 1: 读取 HLIT / HDIST / HCLEN

首先读:

```
HLIT   5 bits
HDIST   5 bits
HCLEN   4 bits
```

实际数量不是字段值本身，而是:

```
NLIT   = HLIT + 257
NDIST   = HDIST + 1
NCLEN   = HCLEN + 4
```

因此:

```
Literal/Length code 数量 = 257 ~ 286
Distance code 数量      = 1 ~ 32
Code-Length code 数量   = 4 ~ 19
```

8. Dynamic Huffman Step 2: 先建立 Code-Length Huffman Tree

现在有一个小问题:

我们需要 Huffman tree

↑
但是 Huffman tree 本身也是压缩的

因此 DEFLATE 引入第三个 alphabet:

Code Length Alphabet

只有 19 个 symbol:

0 ~ 18

Decoder 读取:

$(HLEN + 4) \times 3 \text{ bits}$

每个值代表一个 code length。

但是 symbol 顺序非常特殊，不是:

0, 1, 2, 3, 4...

而是固定:

16, 17, 18, 0, 8, 7, 9, 6, 10, 5, 11, 4, 12, 3, 13, 2, 14, 1, 15

例如:

```
read:
symbol 16 → length 2
symbol 17 → length 3
symbol 18 → length 3
symbol 0  → length 2
...
```

然后通过这些 length 建立:

Code-Length Huffman Table

9. 为什么只传 Huffman Code Length?

这是理解 DEFLATE 的一个关键点。

DEFLATE 使用的是:

Canonical Huffman Code (规范 Huffman 码)

也就是说：

只要知道：

Symbol $\rightarrow$ Code Length

就能唯一重建：

Symbol $\rightarrow$ Huffman Code

不需要把 Huffman tree 的左右节点逐个传输。

RFC 给出的重建过程是：

① 统计每种 bit length 有多少 code

```
bl_count[length]++
```

② 计算每个 length 的第一个 Huffman code

```
code = 0
```

```
for bits = 1 .. MAX_BITS:  
    code = (code + bl_count[bits-1]) << 1  
    next_code[bits] = code
```

③ 按 symbol 顺序分配 code

```
code[symbol] = next_code[length]  
next_code[length]++
```

这就是 Canonical Huffman reconstruction。

对于 Decoder/硬件来说，这意味着：

```
bitstream  
↓  
Code Length  
↓  
Canonical Huffman Builder  
↓  
Decode LUT / Decode Tree
```

而不是接收真正的 tree topology。

10. Dynamic Huffman Step 3: 解码真正两个 Huffman 表的 Code Length

现在有了：

Code-Length Huffman Decoder

它开始解 bitstream。

目标是得到：

NLIT 个 Literal/Length code lengths
+
NDIST 个 Distance code lengths

总共：

NLIT + NDIST

个 length。

Code-Length alphabet 的 symbol 含义如下：

Symbol	含义
0 – 15	Huffman code length = 0 – 15
16	重复前一个 length 3 – 6 次
17	重复 length=0, 3 – 10 次
18	重复 length=0, 11 – 138 次

例如：

8
16 + extra bits = 2

假设：

symbol 8

表示：

```
length = 8
```

然后 symbol 16 表示:

```
repeat previous length
```

extra = 2:

```
repeat = 3 + 2 = 5
```

最后得到:

```
8 8 8 8 8 8
^ +---5 copies---+
```

11. Dynamic Huffman Step 4: 建立两个真正的数据 Huffman Decoder

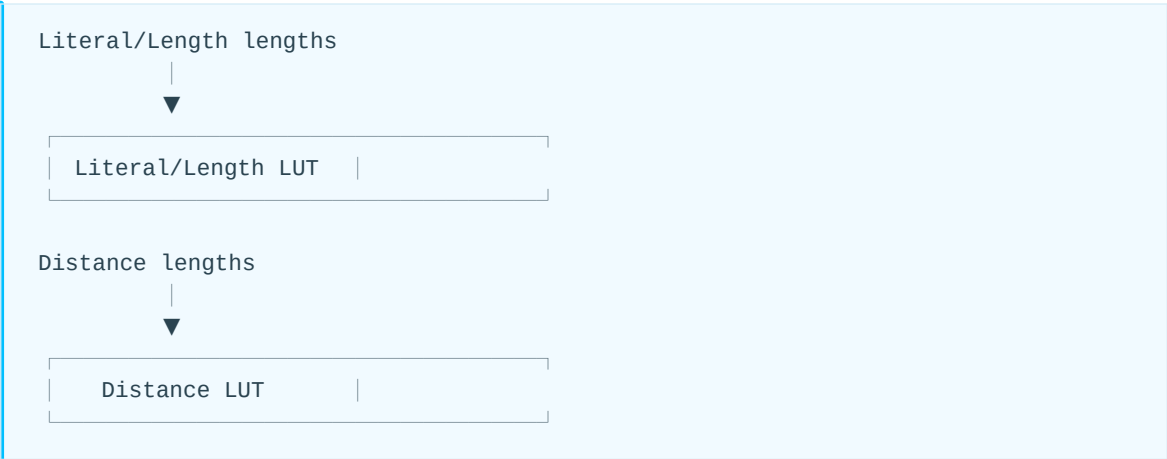
得到:

```
Literal/Length Code Length Array
0 ... NLIT-1
```

和:

```
Distance Code Length Array
0 ... NDIST-1
```

之后分别调用 Canonical Huffman Builder:



到这里 Dynamic Huffman block 的“header 解码”才算完成。

后面的 data decoding 和 Fixed Huffman 完全一样。

Fixed 和 Dynamic compressed block 的差别就在于 Huffman codes 如何定义/获得。

12. Step 5: 真正开始解 Data

现在不论：

```
BTYPE = 01 Fixed
```

还是：

```
BTYPE = 10 Dynamic
```

最终都会进入同样的 loop：

```
while true:
    symbol = decode_literal_length()

    if symbol <= 255:
        output(symbol)

    else if symbol == 256:
        End Of Block
        break

    else:
        decode length
        decode distance
        history copy
```

这实际上就是整个 DEFLATE Decoder 最核心的循环。

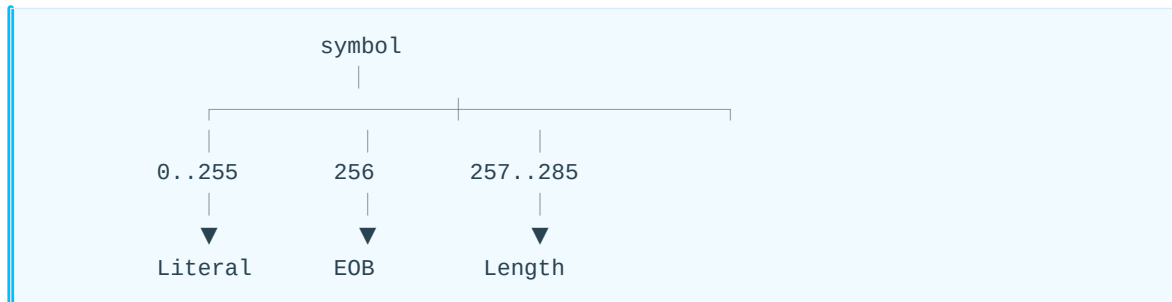
13. Literal/Length Huffman Decoder 输出什么？

Literal/Length alphabet 可以看成：

0	—————	255
	Literal	
256		
	End Of Block	
257	—————	285
	Length Code	

RFC 将 literal 和 length 合并到了同一个 Huffman alphabet。

因此 Decoder 解出 symbol 后，第一件事情就是分类：



14. Case A: symbol = 0~255

这是最简单的情况。

例如：

```
symbol = 0x41
```

那么：

```
output = 'A'
```

同时它也进入 LZ77 history buffer：

```
output stream:
... X X X A
      ↑
    history
```

因为后面的 distance 可能引用它。

15. Case B: symbol = 256

```
symbol = 256
```

表示：

```
End Of Block
```

Decoder 停止当前 Huffman data loop。

然后检查 block header 中之前保存的：

```
BFINAL
```

如果：

```
BFINAL = 0
```

继续读：

```
next block
```

如果：

```
BFINAL = 1
```

则：

```
DEFLATE stream finished
```

16. Case C: symbol = 257~285 —— Length

这里 symbol 本身通常并不直接等于 length。

而是：

```
Length =  
    BaseLength[length_code]  
    +  
    ExtraBits
```

例如 RFC 定义：

```
code 257 → length 3  
code 258 → length 4  
...  
code 264 → length 10  
  
code 265 → length 11~12  
code 266 → length 13~14  
  
...  
  
code 284 → length 227~257  
  
code 285 → length 258
```

例如 Decoder 解出：

```
length_code = 268
```

RFC 定义:

```
base_length = 17  
extra_bits = 1
```

如果额外 bit:

```
1
```

则:

```
length = 17 + 1  
        = 18
```

17. 接着 decode Distance

Length 后面必定跟一个:

```
Distance Huffman Symbol
```

Distance symbol 范围有效值是:

```
0 ... 29
```

对应:

```
distance = 1 ... 32768
```

同样:

```
Distance =  
    BaseDistance[distance_code]  
    +  
    ExtraBits
```

例如:

```
distance_code = 6
```

RFC 表:

```
base distance = 9
extra bits    = 2
```

如果：

```
extra = 2
```

那么：

```
distance = 9 + 2
          = 11
```

18. Step 6: LZ77 History Copy

此时 Decoder 已经得到：

```
Length  = L
Distance = D
```

含义是：

从当前 output position 向前 D bytes，复制 L bytes。

也就是：

```
src = write_pos - distance

for i = 0 .. length-1:
    output[write_pos] = output[src]
    src++
    write_pos++
```

这就是 DEFLATE Decoder 里面的 LZ77 reconstruction。

19. 一个非常重要的问题：Source 和 Destination 可以重叠

例如 output 当前已有：

```
... X Y
```

现在收到：

```
length  = 5
distance = 2
```

源位置：

```
← 2 bytes
```

开始逐 byte copy：

```
初始：
X Y
↑
src
```

copy：

```
X Y X
```

然后：

```
Y X Y
```

继续：

```
X Y X Y X
```

最终新增：

```
X Y X Y X
```

RFC 明确允许这种 overlapping copy。

这对于硬件设计非常关键。

不能简单理解成：

```
memcpy(dst, src, length)
```

因为：

```
length > distance
```

时，刚生成的数据本身会马上成为后续 copy 的 source。

20. 因此 DEFLATE Decoder 的 LZ77 本质上只需要一个 32 KiB history

RFC 1951:

```
distance max = 32768 bytes
length max   = 258 bytes
```

所以 Decoder 可以维护:

32 KiB Circular History Buffer

write_ptr



... previously decoded bytes ...



read_ptr = write_ptr - distance

每输出一个 byte:

```
history[write_ptr] = byte
write_ptr = (write_ptr + 1) % 32768
```

遇到 match:

```
read_ptr = write_ptr - distance

repeat length times:
    byte = history[read_ptr]
    history[write_ptr] = byte

    output(byte)

    read_ptr++
    write_ptr++
```

这天然支持 overlap。

21. 一条 compressed block 最终可以抽象成 Token Stream

经过 Huffman Decoder 之后, 其实可以暂时忘掉 Huffman。

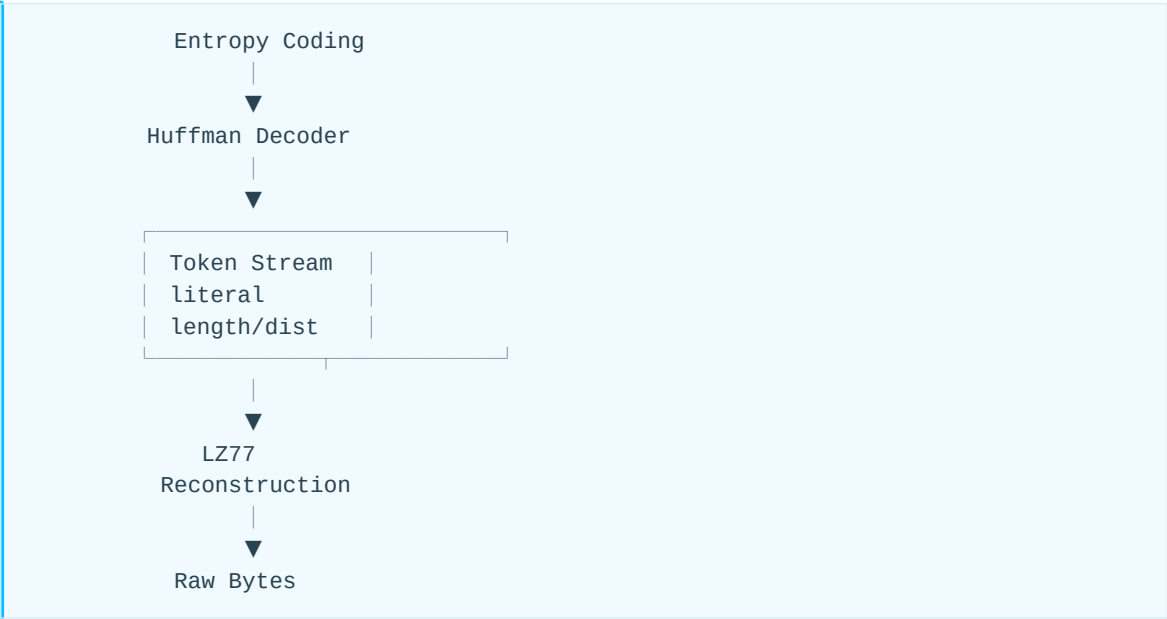
它产生的逻辑 token 就是:

```
Literal
Literal
Literal
<Length, Distance>
Literal
<Length, Distance>
...
EOB
```

例如：

```
'H'
'e'
'l'
'l'
'o'
' '
<length=12, distance=6>
EOB
```

所以从体系结构角度：



这个抽象非常有用。

22. 完整 Decoder 伪代码

如果把 RFC 1951 写成比较接近实现的形式：

C

```
do {

    BFINAL = read_bits(1);
    BTYPE  = read_bits(2);

    switch (BTYPE) {

    case 0:      // stored block

        byte_align();

        LEN  = read_u16();
        NLEN = read_u16();

        check(LEN == (~NLEN & 0xffff));

        for (i = 0; i < LEN; i++) {
            b = read_byte();
            output(b);
            history_put(b);
        }

        break;

    case 1:      // fixed Huffman

        litlen_table = fixed_litlen_table;
        dist_table   = fixed_dist_table;

        decode_compressed_block();
        break;

    case 2:      // dynamic Huffman

        HLIT = read_bits(5) + 257;
        HDIST = read_bits(5) + 1;
        HCLEN = read_bits(4) + 4;

        //-----
        // build code-length Huffman table
        //-----

        read_code_length_lengths();

        cl_table =
            build_canonical_huffman(cl_lengths);

        //-----
        // decode litlen/dist code lengths
        //-----

        decode_code_lengths(
            cl_table,
```

```
        litlen_lengths,  
        dist_lengths);  
  
    litlen_table =  
        build_canonical_huffman(litlen_lengths);  
  
    dist_table =  
        build_canonical_huffman(dist_lengths);  
  
    //-----  
    // decode payload  
    //-----  
  
    decode_compressed_block();  
  
    break;  
  
default:    // BTYPE == 3  
  
    ERROR;  
}  
  
} while (!BFINAL);
```

其中:

C

```
decode_compressed_block()
{
    while (1) {

        symbol = huffman_decode(litlen_table);

        //-----
        // Literal
        //-----

        if (symbol < 256) {

            output(symbol);
            history_put(symbol);

            continue;
        }

        //-----
        // End Of Block
        //-----

        if (symbol == 256)
            break;

        //-----
        // LZ77 Length
        //-----

        length =
            length_base[symbol]
            + read_bits(length_extra[symbol]);

        //-----
        // LZ77 Distance
        //-----

        dcode =
            huffman_decode(dist_table);

        distance =
            dist_base[dcode]
            + read_bits(dist_extra[dcode]);

        //-----
        // History Copy
        //-----

        for (i = 0; i < length; i++) {

            b = history_get(distance);

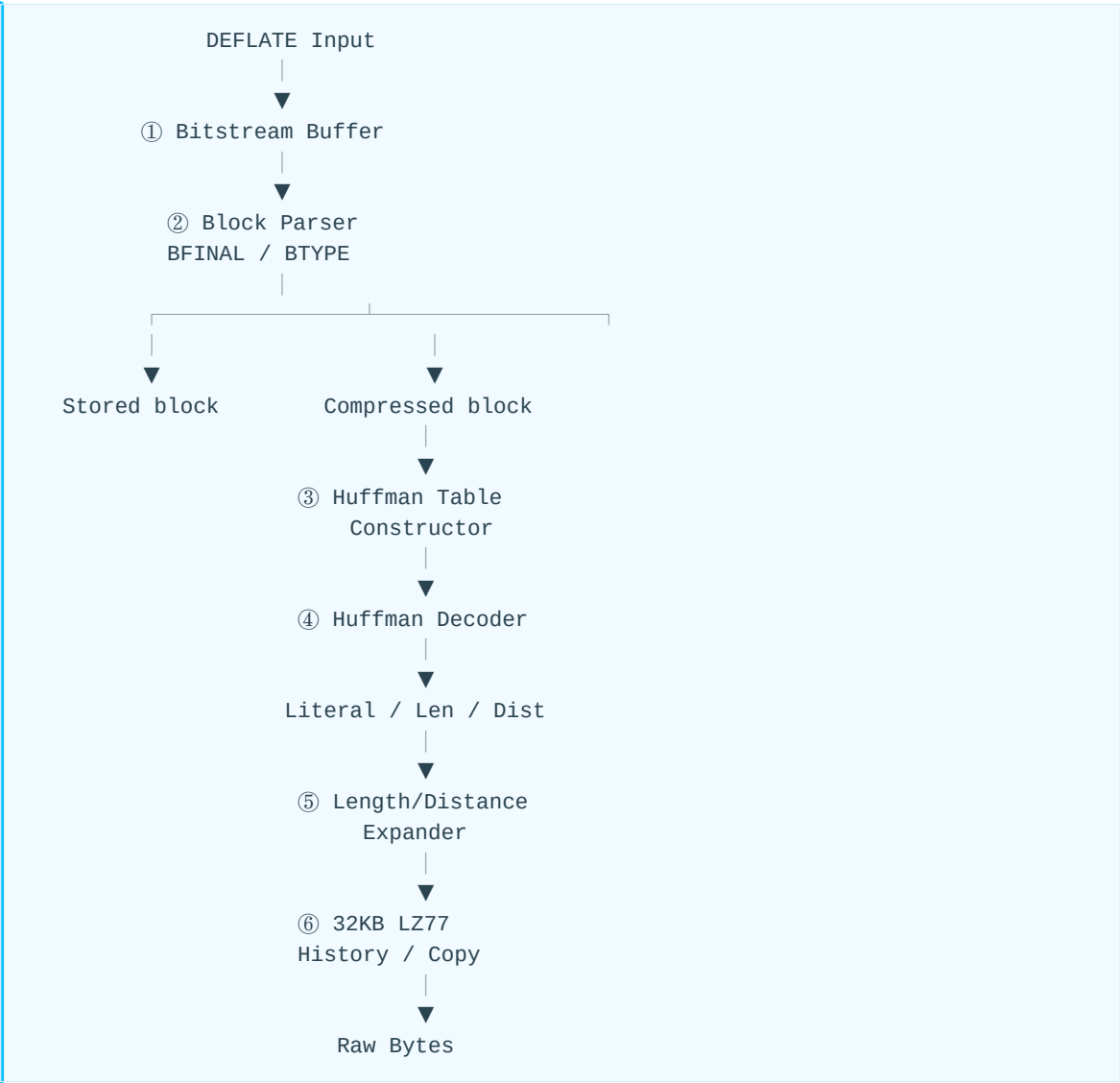
            output(b);
            history_put(b);
        }
    }
}
```

```
}  
}
```

这个伪代码已经非常接近一个真正可实现的 RFC 1951 Decoder。

23. 从硬件角度，可以拆成 6 个主要模块

如果最终目的是做 PNG/DEFLATE 硬件，建议不要把 DEFLATE 看成一个“大 decoder”，而拆成：



其中复杂度并不平均。

Decoder 真正需要重点处理的是三块

① Variable-length Huffman decoding

```
1 symbol = 1~15 bits
```

要解决：

- bit alignment
- lookup table
- variable consume length
- dynamic table rebuild

② Dynamic Huffman header

本质是：

```
Huffman
  ↓
decode code lengths
  ↓
build Huffman
```

状态机比 Fixed Huffman 明显复杂。

③ LZ77 overlap copy

特别是：

```
distance < length
```

例如：

```
distance = 1
length   = 258
```

意味着：

```
A → AAAAAA...
```

对 history SRAM 的读写带宽、RAW dependency 和 throughput 设计非常重要。

24. 一个非常重要的 Bit-order 陷阱

RFC 1951 的 bit order 非常容易让实现者困惑：

```

普通 fixed-width fields
    ↓
LSB first

Huffman canonical code
    ↓
逻辑上 MSB → LSB 定义

bitstream 中
    ↓
看起来是 bit-reversed Huffman code

```

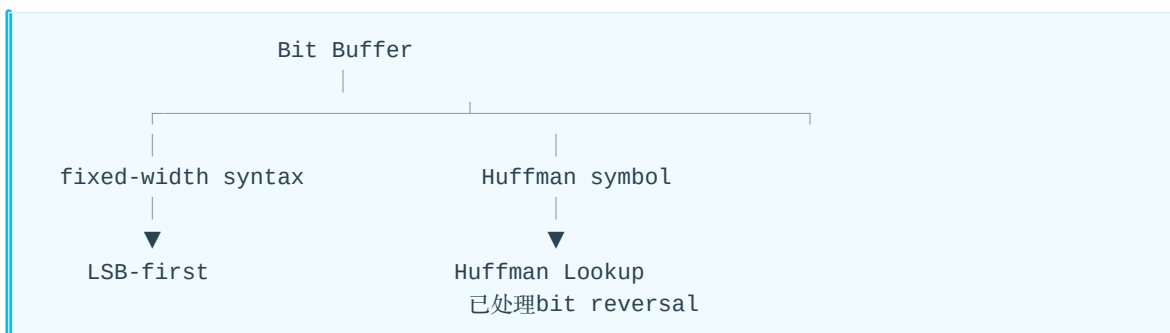
RFC 3.1.1 明确说明了这个区别。

另外 RFC 3.2.5 关于 length/distance extra bits 有一处长期容易引起误解的描述；RFC Editor 有一条 Reported Technical Erratum 7764，指出 extra bits 应按照 DEFLATE 的普通非-Huffman 字段规则 LSB-first 处理。

参考：

- <https://www.rfc-editor.org/rfc/rfc1951.html>
- <https://www.rfc-editor.org/errata/eid7764>

所以实际实现 Decoder 时，更建议统一形成下面的脑图：



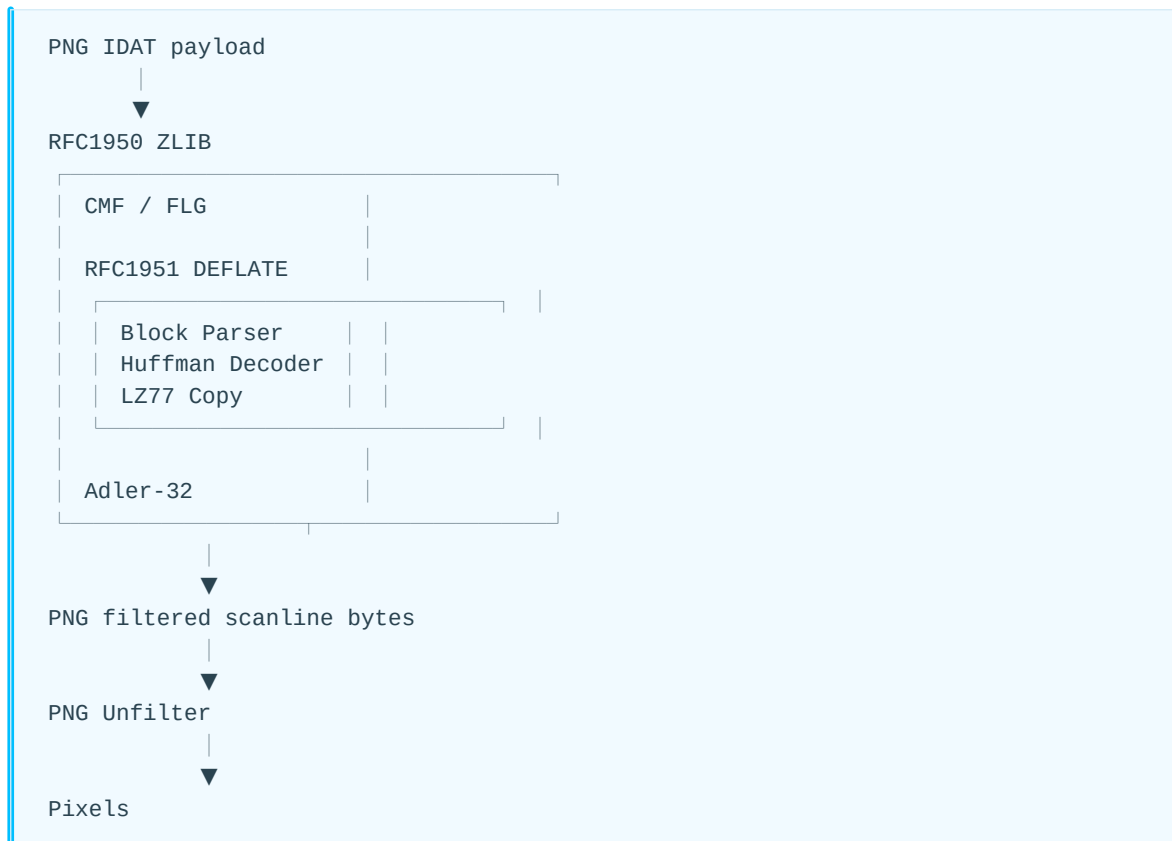
这样不容易在 RTL 里面到处插 bit_reverse()。

25. 最后压缩成一句 Decoder 算法

RFC 1951 DEFLATE 解码其实就是：

逐 block 读取 BTYPE → 得到或构建 Huffman 表 → Huffman 解出 Literal / Length token → Literal 直接输出，Length 再解 Distance → 从最多 32 KiB 的历史输出中执行 LZ77 回拷 → 遇到 symbol 256 结束 block → BFINAL=1 时结束整个 DEFLATE stream。

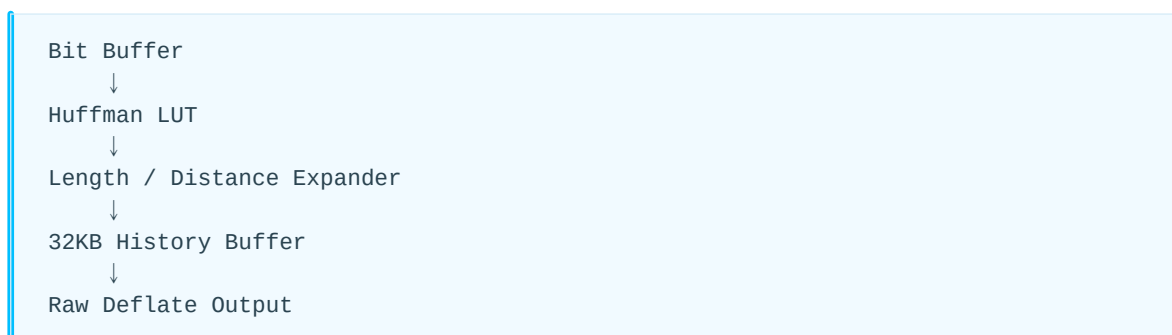
如果进一步用 PNG Decoder 的层次来看，则位置关系非常清楚：



所以严格地说，RFC 1951 Decoder 的输出还不是 PNG pixel，也不是 PNG filter reconstruction 后的数据，而是 PNG filter 处理前的 scanline byte stream；PNG 的 Sub / Up / Average / Paeth unfilter 是下一层的事情。

26. 硬件实现时建议继续展开的主题

如果后续从硬件实现角度继续分析，可以沿着下面的链路拆解：



重点问题包括：

1. Huffman Decoder 如何做到每 cycle 解一个 symbol。
2. Dynamic Huffman 表如何快速构建。
3. Huffman LUT 应选择一级 LUT、两级 LUT 还是 tree walk。
4. LZ77 overlap copy 为什么会形成 RAW dependency。

5. distance=1、length=258 等极端 case 如何维持吞吐率。
6. 32 KiB history buffer 如何映射到 SRAM / BRAM。
7. DEFLATE 输出如何与 PNG scanline buffering、Unfilter 模块衔接。

06-1 PNG Huffman 编码详解

把 Huffman 编码（Huffman Coding，霍夫曼编码）理解成一句话：

出现频率高的符号用短码，出现频率低的符号用长码，从而降低平均编码长度。

它是 PNG 里的 DEFLATE 压缩非常核心的一环。在 PNG → zlib → DEFLATE 的链路中，Huffman 负责把 LZ77 产生的符号进一步编码成 bitstream。

1. 先看最简单的例子

假设一段数据只有 4 种符号：

符号	出现次数	概率
A	50	50%
B	25	25%
C	15	15%
D	10	10%

如果固定长度编码，因为有 4 个符号，每个符号都需要 2 bit：

A = 00
B = 01
C = 10
D = 11

100 个符号总共：

$100 \times 2 = 200 \text{ bit}$

但 A 出现了一半，却仍然每次都要花 2 bit。

Huffman 的思路就是：

A 很常见 → 给短码
D 很少见 → 可以给长码

例如可能得到：

```
A = 0
B = 10
C = 110
D = 111
```

那么平均长度：

```
A: 50 × 1 = 50
B: 25 × 2 = 50
C: 15 × 3 = 45
D: 10 × 3 = 30
-----
总计 = 175 bit
```

从固定编码的：

```
200 bit
```

下降到：

```
175 bit
```

这就是 Huffman 压缩的本质。

2. Huffman 编码树怎么构造？

Huffman 的关键不是“随便给高频符号短码”，而是有一个确定的建树算法。

还是：

```
A = 50
B = 25
C = 15
D = 10
```

首先把权重从小到大排列：

```
D 10
C 15
B 25
A 50
```

第一步：合并最小的两个

```
D 10 + C 15 = 25
```

变成：

```
25 (D+C)
25 B
50 A
```

第二步：再合并最小的两个

```
25 + 25 = 50
```

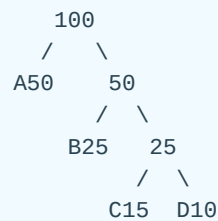
得到：

```
50 ((D+C)+B)
50 A
```

第三步：合并

```
50 + 50 = 100
```

最终得到一棵树：



假定：

```
左分支 = 0
右分支 = 1
```

那么：

```
A = 0
B = 10
C = 110
D = 111
```

这就是 Huffman code。

3. 为什么解码时不会歧义？

这是 Huffman 非常重要的一个性质：

Huffman code 是一种 Prefix Code，前缀码。

也就是说：

任何一个完整码字，都不会是另一个码字的前缀。

例如：

```
A = 0
B = 10
C = 110
D = 111
```

不会存在：

```
B = 10
另一个符号 = 101
```

因为如果 10 已经代表 B，那么读到：

```
10
```

解码器就已经确定是 B。

例如收到 bitstream：

```
0101101110
```

从左向右走 Huffman tree：

```
0   → A
10  → B
110 → C
111 → D
0   → A
```

所以：

```
0101101110
↓
A B C D A
```

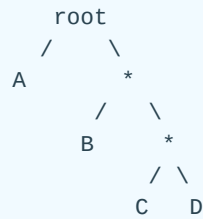
不需要分隔符。

4. 从树的角度理解解码

Huffman 解码器可以把每个 bit 看成一次树遍历：

```
bit = 0 → 左走  
bit = 1 → 右走
```

例如：



输入：

```
110
```

过程：

```
root
↓ 1
*
↓ 1
*
↓ 0
C
```

到达叶子节点：

```
symbol = C
```

然后回到 root，继续解析下一个 symbol。

所以最朴素的 Huffman decoder 可以写成：

```
node = root

while not leaf(node):
    bit = read_bit()
    if bit == 0:
        node = node.left
    else:
        node = node.right

output(node.symbol)
```

不过实际软件和硬件通常不会真的一 bit 一 bit 地走树，因为性能太低。

5. Huffman 真正优化的是什么？

Huffman 不是直接让：

压缩后长度 = 最小

而是在给定符号概率情况下，构造一个非常好的 整数 bit 长度前缀码。

理想的信息量是：

$$I(x) = -\log_2 P(x)$$

例如某符号概率：

$P = 1/2$

理想编码长度：

$$-\log_2(1/2) = 1\text{bit}$$

如果：

$P = 1/4$

则：

$$-\log_2(1/4) = 2\text{bit}$$

如果：

$P = 1/8$

则：

$$-\log_2(1/8) = 3\text{bit}$$

因此理想情况：

```
概率高
↓
 $-\log_2(P)$  小
↓
编码短
```

这就是信息论背景。

6. Huffman 编码的是“符号”，不是原始 bit

这点和 PNG / DEFLATE 非常相关。

很多人第一次理解 DEFLATE 时会想成：

```
原始字节
↓
Huffman
↓
bitstream
```

其实 DEFLATE 更接近：

```
原始数据
  ↓
  ▼
LZ77
  |
  ├── Literal
  └── <Length, Distance>
      ↓
      Huffman
      ↓
    bitstream
```

例如原始字符串：

```
ABCABCABCABC
```

LZ77 可能表示成：

```
A
B
C
<length=9, distance=3>
```

然后 Huffman 并不是直接编码：

```
ABCABCABCABC
```

而是编码：

```
Literal A  
Literal B  
Literal C  
Length symbol  
Distance symbol
```

所以二者分工不同：

```
LZ77  
解决：重复字符串  
  
Huffman  
解决：符号出现概率不均匀
```

这两个组合起来，就是 DEFLATE 的核心。

7. DEFLATE 为什么有两套 Huffman Tree?

这点对理解 PNG 很重要。

DEFLATE 中主要有：

Literal/Length Huffman Tree

编码：

```
Literal 0~255  
End-of-block = 256  
Length codes = 257~285
```

也就是：

```
0 ----- 255  
| literal bytes  
|  
256  
| End Of Block  
|  
257 ----- 285  
| Length codes
```

另外还有：

Distance Huffman Tree

专门编码:

Distance

所以一个 LZ77 match:

<Length, Distance>

最终变成:

Length symbol
↓
Literal/Length Huffman

Distance symbol
↓
Distance Huffman

然后有些 Length / Distance 还需要追加:

extra bits

例如:

Length Symbol
+ Length Extra Bits

Distance Symbol
+ Distance Extra Bits

因此一个 match 的 DEFLATE 表达大致是:



8. Fixed Huffman 和 Dynamic Huffman

这也是 DEFLATE 里最关键的差别。

DEFLATE block 有三种主要形式：

```
BTYP E = 00
Stored / Uncompressed

BTYP E = 01
Fixed Huffman

BTYP E = 10
Dynamic Huffman
```

8.1 Fixed Huffman

编码长度由 RFC 1951 直接规定。

例如 Literal/Length：

```
0 - 143    → 8 bits
144 - 255  → 9 bits
256 - 279  → 7 bits
280 - 287  → 8 bits
```

也就是说：

```
不需要发送 Huffman tree
```

Decoder 天生就知道。

优点：

```
简单
没有 tree header overhead
```

缺点：

```
不能适应当前数据概率
```

8.2 Dynamic Huffman

Dynamic Huffman 则是：

```
Encoder 分析当前 block 的 symbol frequency
      ↓
    建 Huffman tree
      ↓
    得到 code lengths
      ↓
    把 code lengths 写进 bitstream
      ↓
Decoder 重建 Huffman table
```

所以 Dynamic Huffman 本身还有一部分 overhead。

只有当：

```
节省的数据 bits
>
描述 Huffman tree 的 bits
```

才划算。

9. 一个特别重要的概念：实际传输的是 Code Length

DEFLATE 的 Dynamic Huffman 并不会真的发送：

```
A = 010
B = 11101
C = ...
```

它主要发送的是：

```
每个 symbol 的 code length
```

例如：

```
symbol A length = 1
symbol B length = 2
symbol C length = 3
symbol D length = 3
```

然后 Decoder 根据这些长度重新构造 Huffman codes。

为什么可以这样？

因为 DEFLATE 使用：

Canonical Huffman Code，规范 Huffman 编码

这是理解 DEFLATE decoder 非常关键的一步。

10. Canonical Huffman 是什么？

假设只知道：

```
A length = 1  
B length = 2  
C length = 3  
D length = 3
```

Canonical Huffman 有确定的规则，可以唯一构造：

```
A = 0  
B = 10  
C = 110  
D = 111
```

所以不需要把整棵树发送过去。

只需要发送：

```
code lengths
```

Decoder 就可以重建完全相同的 code table。

这对 DEFLATE 非常重要：

```
普通 Huffman:  
发送 tree 可能很复杂  
  
Canonical Huffman:  
只发送 length  
↓  
decoder 自动恢复 code
```

因此更准确地说，DEFLATE 的 Dynamic Huffman 过程是：

```
Frequency
  ↓
Huffman tree
  ↓
Code lengths
  ↓
Canonical Huffman
  ↓
Actual codes
```

Decoder:

```
Code lengths
  ↓
Canonical Huffman reconstruction
  ↓
Decode table
```

11. 为什么 DEFLATE 还要“再用一个 Huffman” 编码 Huffman tree?

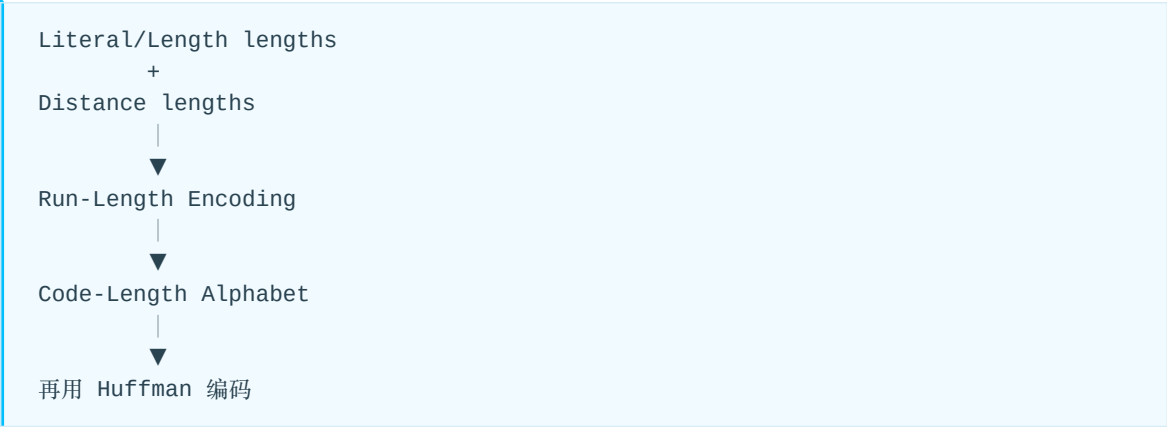
Dynamic Huffman 有一个非常巧妙的设计。

需要描述:

```
Literal/Length code lengths
+
Distance code lengths
```

但如果直接把大量 length 写进去，仍然很浪费。

于是 DEFLATE 又做了一层:



也就是说，有 Huffman 编码 Huffman 的描述信息。

DEFLATE 里的 code-length alphabet 是：

- 0~15 → actual code length
- 16 → repeat previous length 3~6 times
- 17 → repeat zero 3~10 times
- 18 → repeat zero 11~138 times

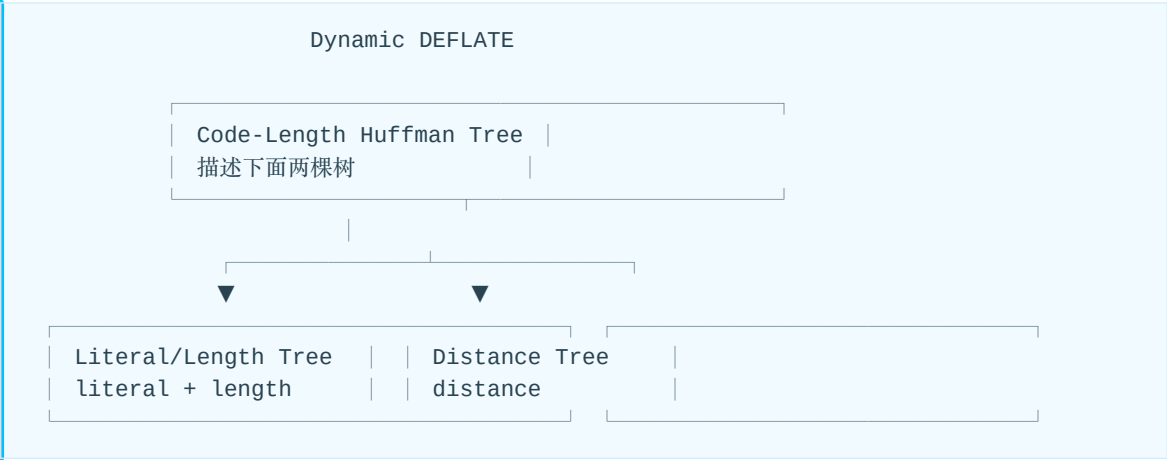
然后这些：

0~18

本身又建立一棵：

Code Length Huffman Tree

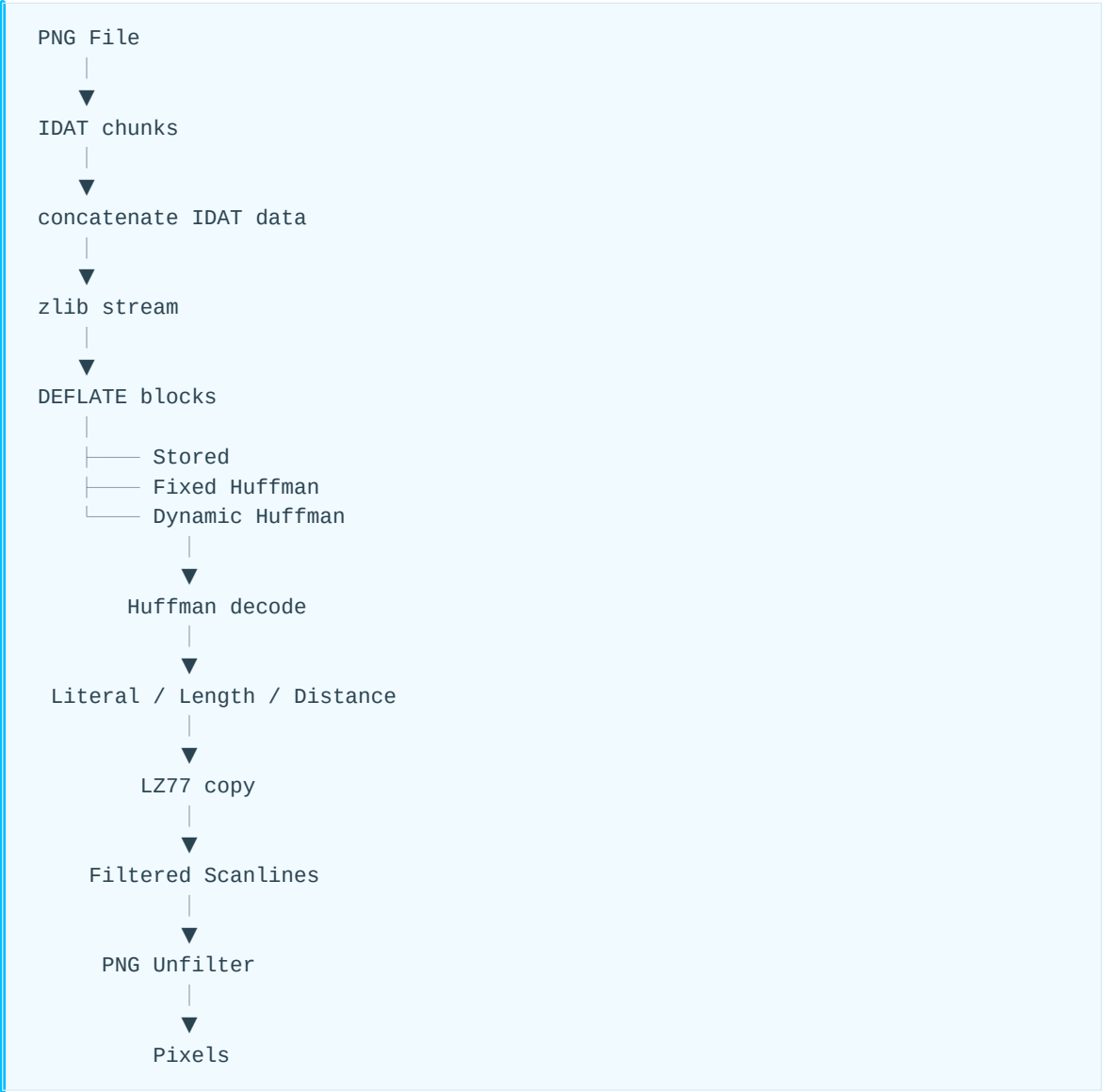
所以 Dynamic DEFLATE 实际上存在 三类 Huffman 表：



这是 RFC 1951 解码最容易第一次看懵的地方。

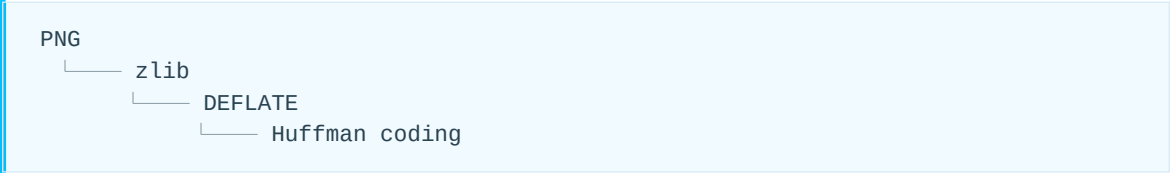
12. 从 PNG Decoder 的角度串起来

PNG 解码时:



所以在 PNG 里，Huffman 并不是 PNG 独有算法。

它属于:



对应标准关系是:

PNG	→ PNG Specification / ISO/IEC 15948
zlib	→ RFC 1950
DEFLATE	→ RFC 1951
Huffman	→ DEFLATE 内部编码机制

13. 最需要建立的一个直觉

可以把 DEFLATE 理解成两级压缩：



所以：

LZ77 消除 “空间上的重复”， Huffman 消除 “符号概率上的冗余”。

这句话基本抓住了 DEFLATE 的核心。

14. 工程化定义总结

Huffman 编码：

根据符号出现概率构造一棵二叉前缀码树，使高频符号拥有较短码字、低频符号拥有较长码字，从而降低平均编码长度；DEFLATE 使用 canonical Huffman code，并分别编码 Literal/Length 与 Distance symbols。

对于 DEFLATE Decoder，下一步最关键的是理解：

Canonical Huffman 如何只根据 code length 计算出实际 Huffman code。

这是从“理解 Huffman”进入“真正能写 DEFLATE decoder”的关键一步。

07. Scanline Filter 与重建

1. Filter 的目标

相邻像素往往相似。若直接压缩 RGB byte，数值本身分布可能很散；若存“当前值减预测值”，大量结果会靠近0，并形成重复模式，更利于 LZ77 和 Huffman。

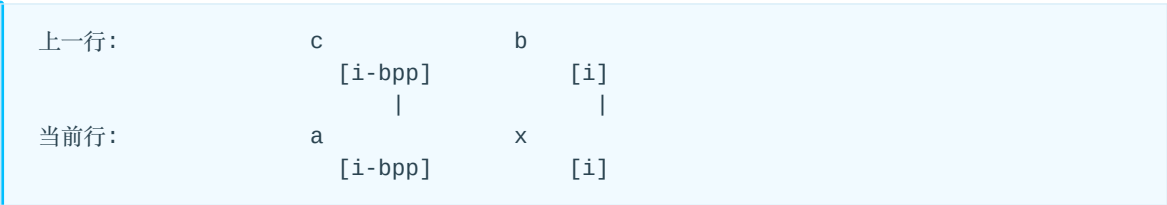
Filter 是逐行可逆变换。每行可独立选择五种 Filter Type 之一。

定义：

- x: 编码前当前 byte；
- f: Filtered byte；
- a: 当前行左侧 bpp 处已恢复 byte；
- b: 上一行同位置 byte；
- c: 上一行左侧 bpp 处 byte。

缺失的 a、b、c 取0。运算对256取模。

位置关系如下，x 是当前要编码或重建的 byte；a、b、c 都是已经可用的相邻 byte：



其中 a 与 x 相隔 bpp 个 byte，b 位于上一行同一 byte 位置，c 是 a 的上方；并非“左侧一个 byte”或“左上一个 byte”，因为一个像素可能包含多个 byte。

2. 五种 Filter

Type	名称	编码 f	解码 x
0	None	x	f
1	Sub	x-a	f+a
2	Up	x-b	f+b
3	Average	x-floor((a+b)/2)	f+floor((a+b)/2)
4	Paeth	x-Paeth(a,b,c)	f+Paeth(a,b,c)

所有结果最终只保留低8 bit。

2.1 None

没有预测依赖，Filtered byte 就是原 byte。它适合已经无明显空间相关性的数据，也常用于最小实现或测试。

2.2 Sub

预测当前 byte 等于同一行前一个像素对应通道：

```
x[i] = f[i] + x[i-bpp]
```

前 bpp 个 byte 的 $\alpha=0$ 。

2.3 Up

预测当前 byte 等于上一行相同位置：

```
x[i] = f[i] + previous[i]
```

第一行或一个 Adam7 Pass 的第一行， $b=0$ 。

2.4 Average

```
predictor = floor((a+b)/2)
```

必须先用足够宽的整数计算 $a+b$ ，避免 8-bit 溢出后再除 2。

2.5 Paeth

Paeth 用 a 、 b 、 c 估计局部二维平面。

```
p = a + b - c
pa = abs(p - a)
pb = abs(p - b)
pc = abs(p - c)

if pa <= pb and pa <= pc: return a
if pb <= pc: return b
return c
```

相等时优先级是 a 、再 b 、再 c 。中间值必须使用有符号且足够宽的整数。

3. 逐行反滤波算法

```
filter = input_byte()
require(filter in 0..4)

for i = 0 .. row_bytes-1:
    f = input_byte()
    a = current[i-bpp] if i >= bpp else 0
    b = previous[i]    if previous exists else 0
    c = previous[i-bpp] if previous exists and i >= bpp else 0

    predictor = select(filter, a, b, c)
    current[i] = (f + predictor) & 0xFF

consume_or_output(current)
swap(current, previous)
```

Filter Type Byte 不参与反滤波公式，也不是 `current[0]`。

3.1 RGB8 Sub 示例

bpp=3。原始行：

```
10 20 30 | 13 25 29
```

Sub 后：

```
10 20 30 | 03 05 FF
```

第二个像素 B 分量：29-30=-1，模256后为 FF。解码时 FF+30=285，保留低8 bit 得29。

3.2 Up 示例

上一行：

```
10 20 30
```

当前原始行：

```
12 18 35
```

Filtered：

```
02 FE 05
```

这里 FE 表示 -2 的模256形式，不代表实际像素很大。

4. 重建依赖

Filter	行内依赖	上一行依赖
None	无	无
Sub	x[i-bpp]	无
Up	无	previous[i]
Average	x[i-bpp]	previous[i]
Paeth	x[i-bpp]	previous[i]、previous[i-bpp]

因此“解压后得到类似残差”是合理直觉，但这种残差：

- 没有 DCT/IDCT；
- 没有量化；
- 以 byte 为单位；
- Predictor 由每行第一个 byte 指定；
- Sub/Average/Paeth 具有当前行递推依赖。

5. 从重建行到像素

正确顺序是：

```
DEFLATE output
-> identify row and filter byte
-> unfilter bytes
-> unpack samples
-> palette/trNS
-> color conversion
-> output pixels
```

不能在 Unfilter 前拆解 RGB 通道，因为 Filter bpp 与序列化 byte 位置共同定义左邻域。

08. Adam7 原理与解码流程

1. Adam7 是什么？

Adam7 是 PNG 标准定义的隔行扫描（Interlace）方案。

它的核心思想是：

将一张完整图像按照特定的二维采样规则拆分成 7 个 Pass，由稀疏到密集依次编码和传输；解码时再将这 7 个 Pass 的像素映射回完整图像。

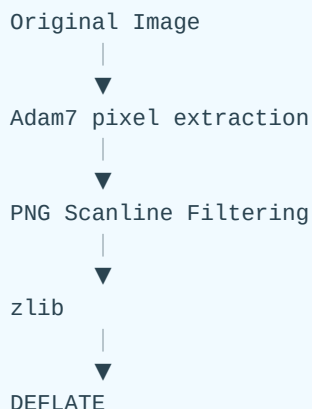
Adam7 的主要目的不是提高压缩率，而是支持 渐进式显示（Progressive Rendering）。

在网络传输场景中，解码器可以先根据前几个 Pass 快速显示一张粗略的全图，然后随着后续 Pass 到达逐步补充细节。

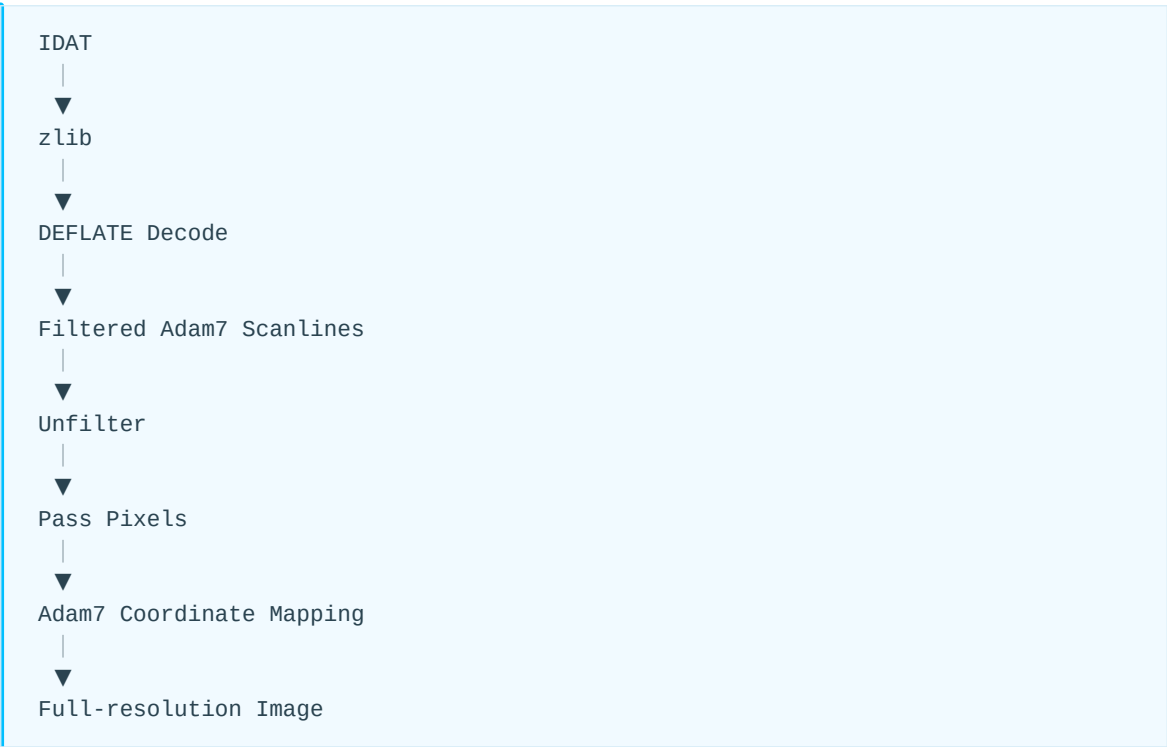
需要特别注意：

- Adam7 不是压缩算法；
- Adam7 不属于 DEFLATE；
- Adam7 不属于 Huffman 或 LZ77；
- Adam7 本质上是一种 像素扫描顺序与图像重排方案。

从 PNG 编码链路看，其位置大致为：

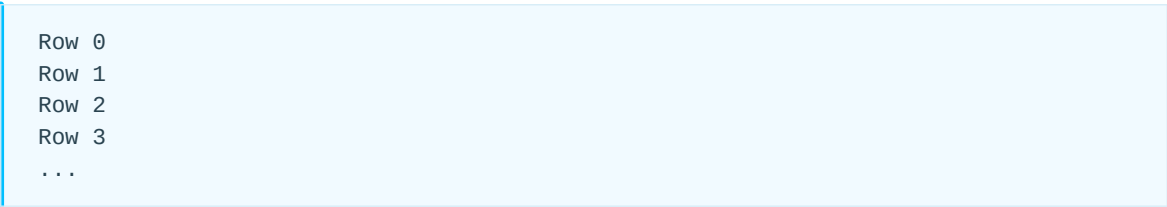


解码方向则相反：



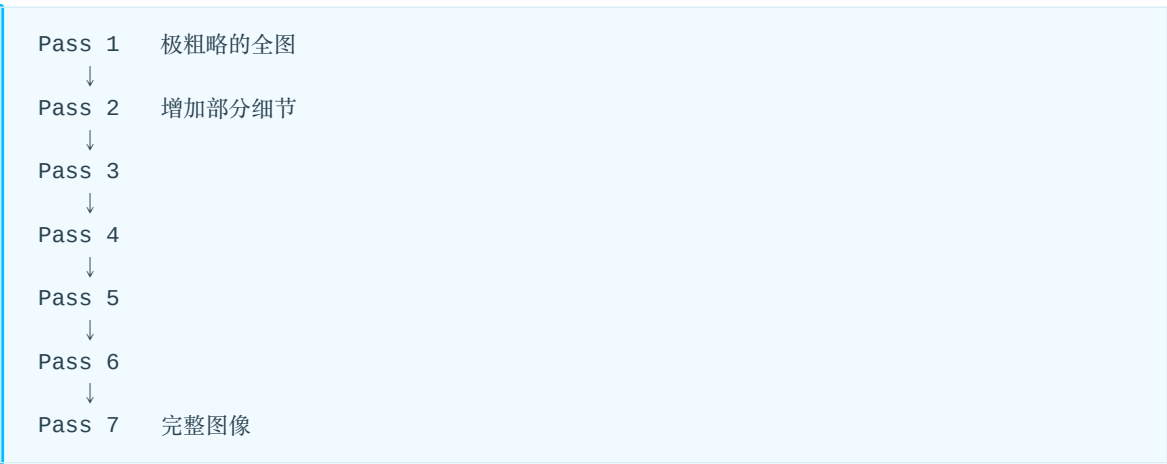
2. 为什么 PNG 需要 Adam7?

普通非隔行 PNG 按照扫描线顺序编码：



如果图片很大，网络传输尚未完成时，解码器往往只能逐行显示图像顶部。

Adam7 改变了像素传输顺序，使得数据逐步覆盖整幅图像：



因此 Adam7 的主要价值是：

在完整文件尚未接收完成时，尽早得到整幅图像的低分辨率预览。

3. Adam7 的 8×8 基本模式

Adam7 的扫描规则可以通过一个 8×8 模式直观表示：

```
1 6 4 6 2 6 4 6
7 7 7 7 7 7 7 7
5 6 5 6 5 6 5 6
7 7 7 7 7 7 7 7
3 6 4 6 3 6 4 6
7 7 7 7 7 7 7 7
5 6 5 6 5 6 5 6
7 7 7 7 7 7 7 7
```

数字表示某个像素在哪个 Pass 中被编码。

例如：

- 1 表示该像素属于 Pass 1；
- 4 表示属于 Pass 4；
- 7 表示直到最后的 Pass 7 才出现。

7 个 Pass 最终恰好覆盖全部像素，并且每个像素只属于一个 Pass。

4. 7 个 Pass 的参数

每个 Adam7 Pass 可以用以下四个参数描述：

- 起始横坐标 x0
- 起始纵坐标 y0
- 水平步长 dx
- 垂直步长 dy

Pass	x0	y0	dx	dy
1	0	0	8	8
2	4	0	8	8
3	0	4	4	8
4	2	0	4	4

Pass	x0	y0	dx	dy
5	0	2	2	4
6	1	0	2	2
7	0	1	1	2

一个 Pass 内的像素坐标满足：

```
x = x0 + n × dx
y = y0 + m × dy
```

其中：

```
n = 0, 1, 2, ...
m = 0, 1, 2, ...
```

5. Pass 1：建立最粗略的图像骨架

Pass 1 参数：

```
x0 = 0
y0 = 0
dx = 8
dy = 8
```

因此选中的像素类似：

```
(0,0)  (8,0)  (16,0) ...
(0,8)  (8,8)  (16,8) ...
...
```

在单个 8×8 区域中：

```
X . . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
```

每个 8×8 区域只保留一个像素，因此 Pass 1 的数据量非常小。

它提供的是整幅图像最粗略的结构。

6. Pass 2

Pass 2 参数：

```
x0 = 4
y0 = 0
dx = 8
dy = 8
```

新增像素类似：

```
. . . . X . . .
```

Pass 1 与 Pass 2 合并后：

```
X . . . X . . .
. . . . . . .
...
```

水平方向的采样密度进一步提高。

7. Pass 3

Pass 3 参数：

```
x0 = 0
y0 = 4
dx = 4
dy = 8
```

增加：

```
第 4 行：
X . . . X . . .
```

前三个 Pass 合起来大致形成：

```
X . . . X . . .
. . . . . . .
. . . . . . .
. . . . . . .
X . . . X . . .
. . . . . . .
. . . . . . .
. . . . . . .
```

此时已经能够形成一个低分辨率的整图预览。

8. Pass 4~7: 逐步填补缺失像素

8.1 Pass 4

```
x0 = 2  
y0 = 0  
dx = 4  
dy = 4
```

其像素位置为:

```
x = 2, 6, 10, ...  
y = 0, 4, 8, ...
```

8.2 Pass 5

```
x0 = 0  
y0 = 2  
dx = 2  
dy = 4
```

其像素位置为:

```
x = 0, 2, 4, 6, ...  
y = 2, 6, 10, ...
```

8.3 Pass 6

```
x0 = 1  
y0 = 0  
dx = 2  
dy = 2
```

其像素位置为:

```
x = 1, 3, 5, 7, ...  
y = 0, 2, 4, 6, ...
```

8.4 Pass 7

```
x0 = 0  
y0 = 1  
dx = 1  
dy = 2
```

Pass 7 覆盖：

```
x = 0, 1, 2, 3, ...  
y = 1, 3, 5, 7, ...
```

也就是剩余的奇数行。

完成 Pass 7 后，所有像素都已经恢复。

最终形成：

```
1 6 4 6 2 6 4 6  
7 7 7 7 7 7 7 7  
5 6 5 6 5 6 5 6  
7 7 7 7 7 7 7 7  
3 6 4 6 3 6 4 6  
7 7 7 7 7 7 7 7  
5 6 5 6 5 6 5 6  
7 7 7 7 7 7 7 7
```

9. 每个 Pass 都应理解为一张独立“小图”

这是理解 Adam7 最重要的概念之一。

不要把 IDAT 中 Adam7 数据简单理解成：

```
Original Row 0 的部分像素  
Original Row 1 的部分像素  
Original Row 2 的部分像素
```

更准确的理解是：

```
Pass 1 Image  
Pass 2 Image  
Pass 3 Image  
...  
Pass 7 Image
```

每一个 Pass 都具有自己的：

- 图像宽度；
- 图像高度；
- 扫描线；
- Filter Byte；
- Previous Scanline；
- 解滤波状态。

例如一张：

```
1920 × 1080
```

的图像，其 Pass 1 大约为：

```
240 × 135
```

因为 Pass 1：

```
dx = 8  
dy = 8
```

因此 Pass 1 自身可以看成一张尺寸为 240×135 的逻辑子图。

10. Adam7 数据在 DEFLATE 解压后的组织形式

DEFLATE 解压完成后，Adam7 数据大致按照以下方式排列：

```
Pass 1  
├─ Filter byte  
├─ Scanline 0 data  
├─ Filter byte  
├─ Scanline 1 data  
└─ ...  
  
Pass 2  
├─ Filter byte  
├─ Scanline 0 data  
├─ Filter byte  
└─ Scanline 1 data  
  
Pass 3  
...  
  
Pass 7  
...
```

也就是说：

每一个 Pass 都被重新组织为自己的扫描线序列，然后再执行 PNG Filter。

Adam7 不是在字节流层面简单地“每隔几个 byte 抽取一次”。

11. Adam7 与 PNG Filter 的关系

PNG Filter 对 Adam7 的每个 Pass 独立工作。

这一点非常关键。

例如 Pass 1 对应的原图行可能是：

```
Original Row 0  
Original Row 8  
Original Row 16  
Original Row 24  
...
```

从原始图像坐标看，它们相隔 8 行。

但是在 Pass 1 内部，它们是：

```
Pass1 Row 0  
Pass1 Row 1  
Pass1 Row 2  
Pass1 Row 3  
...
```

因此，对于 PNG Filter：

Up 表示当前 Pass 中的上一条扫描线，而不是原始图像中的上一行。

例如：

```
Current original row = 16
```

在 Pass 1 中：

```
Up = original row 8 中属于 Pass 1 的像素
```

而不是：

```
original row 15
```

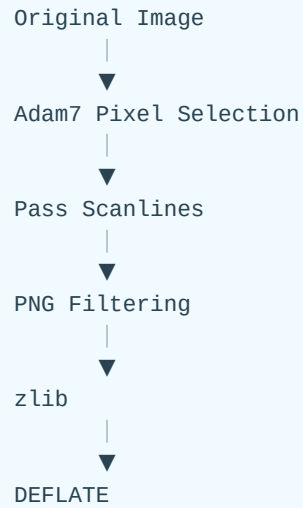
因此硬件或软件实现中必须维护：

```
Previous Scanline of Current Pass
```

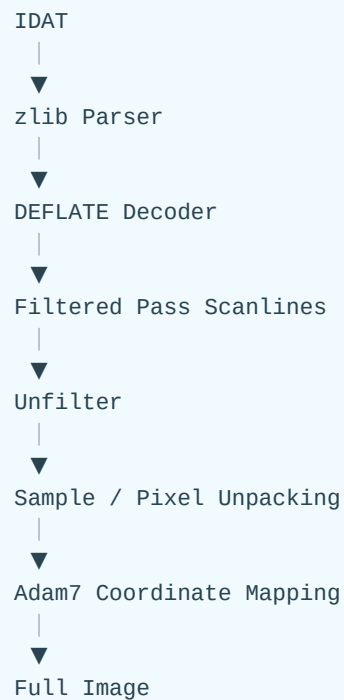
而不是简单维护原图的上一行。

12. Adam7 与 Filter、DEFLATE 的处理顺序

编码流程可以概括为：



解码流程为：



因此在解码方向上：

必须先把当前 Pass 的 scanline 解滤波，再将恢复出来的像素映射回原始图像。

13. IHDR 中如何表示是否使用 Adam7

PNG 的 IHDR 中包含:

```
Interlace method
```

PNG 定义:

```
0 = No interlace
1 = Adam7 interlace
```

因此 PNG Decoder 解析 IHDR 后可以进行:

```
if interlace_method == 0:
    normal scanline decoding
else if interlace_method == 1:
    Adam7 decoding
```

Adam7 是否启用由 PNG 的 IHDR 决定, 与 zlib 或 DEFLATE Header 无关。

14. 如何计算每个 Pass 的宽度和高度

假设原始图像尺寸:

```
Width  = W
Height = H
```

某个 Pass 参数:

```
x_start = x0
y_start = y0
x_step  = dx
y_step  = dy
```

则 Pass 宽度:

$$W_p = \begin{cases} 0, & W \leq x_0 \\ \lceil \frac{W-x_0}{dx} \rceil, & W > x_0 \end{cases}$$

Pass 高度:

$$H_p = \begin{cases} 0, & H \leq y_0 \\ \lceil \frac{H-y_0}{dy} \rceil, & H > y_0 \end{cases}$$

例如:

```
W = 1920
H = 1080
```

Pass 1:

```
x0 = 0
y0 = 0
dx = 8
dy = 8
```

得到:

```
Wp = ceil(1920 / 8)
    = 240

Hp = ceil(1080 / 8)
    = 135
```

因此:

```
Pass 1 Size = 240 × 135
```

15. 8×8 图像中的像素数量

对于恰好 8×8 的图像:

Pass	像素数量
1	1
2	1
3	2
4	4
5	8
6	16
7	32
总计	64

即:

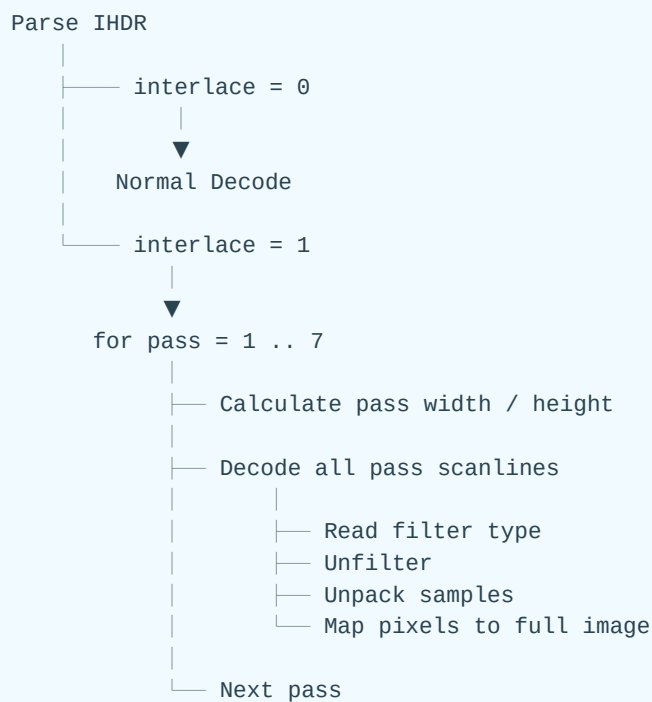
```
1 + 1 + 2 + 4 + 8 + 16 + 32 = 64
```

从这里也可以直观看出 Adam7 的设计思想：

后续 Pass 不断增加大约一倍的新像素，使图像分辨率逐步提升。

16. Adam7 Decoder 的基本算法

Adam7 解码器可以抽象为：



像素映射公式为：

```

dst_x = x0[pass] + pass_x × dx[pass]
dst_y = y0[pass] + pass_y × dy[pass]
  
```

例如 Pass 6：

```

x0 = 1
y0 = 0
dx = 2
dy = 2
  
```

则：

```
Pass (0,0) → Original (1,0)
Pass (1,0) → Original (3,0)
Pass (2,0) → Original (5,0)
Pass (3,0) → Original (7,0)

Pass (0,1) → Original (1,2)
Pass (1,1) → Original (3,2)
...
```

17. Adam7 对硬件 Decoder 的影响

Adam7 算法本身并不复杂，但它会显著增加硬件数据流设计的复杂度。

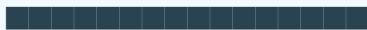
主要挑战如下。

17.1 Frame Buffer 写地址不连续

普通非隔行 PNG 解码输出：

```
Row 0
Row 1
Row 2
...
```

通常能够形成连续写：



而 Adam7 前几个 Pass 的输出类似：



因此需要进行稀疏写入（Scatter Write）。

实际 Frame Buffer 地址为：

```
address =
    base
    + y × stride
    + x × bytes_per_pixel
```

这会降低 DDR Burst 或传统 VDMA 顺序写入的效率。

17.2 Filter Line Buffer 必须是 Pass-local

Filter 的参考行必须是：

```
Previous Scanline of Current Pass
```

例如 Pass 1:

```
Original Row 0  
Original Row 8  
Original Row 16
```

在 Filter 语义中是连续三条扫描线。

因此 Filter Buffer 的管理需要跟随当前 Pass，而不是原始图像 y 坐标。

17.3 每个 Pass 的 Scanline 长度不同

对于较大的图片，各个 Pass 的宽度大致为：

```
Pass 1 : W / 8  
Pass 2 : W / 8  
Pass 3 : W / 4  
Pass 4 : W / 4  
Pass 5 : W / 2  
Pass 6 : W / 2  
Pass 7 : W
```

因此：

```
scanline_bytes
```

会随着 Pass 改变。

硬件控制逻辑需要动态计算：

- Pass Width;
- Pass Height;
- Scanline Bytes;
- Filter Buffer Size;
- Pixel Destination Address。

17.4 Bit Depth 小于 8 时需要特别处理

对于：

- 1-bit grayscale;
- 2-bit grayscale;
- 4-bit grayscale;
- 1/2/4-bit palette PNG;

一个 byte 内可能包含多个 pixel sample。

Adam7 的 Pass 数据会按照该 Pass 自身的像素序列重新进行 sample packing。

因此不能把 Adam7 实现成：

每隔 N 个 byte 取一个 byte

必须明确区分：

Original Pixel Coordinate
↓
Sample
↓
Pass Pixel Sequence
↓
Packed Scanline Bytes

这也是 Adam7 Decoder 实现中容易出错的地方之一。

18. Adam7 在完整 PNG Decoder 中的位置

一个较完整的 PNG 解码数据流可以表示为：



更准确地说，对于 Adam7：

```
DEFLATE output
→ determine current pass / scanline
→ unfilter
→ unpack samples
→ map pass pixel coordinates
→ write full-resolution image
```


19. Adam7 的四个核心结论

19.1 Adam7 不是压缩算法

```
Adam7 ≠ DEFLATE  
Adam7 ≠ Huffman  
Adam7 ≠ LZ77
```

Adam7 属于 PNG 的：

图像扫描顺序与隔行编码机制。

19.2 Adam7 将图像组织成 7 个逻辑子图

```
Pass 1  
Pass 2  
Pass 3  
Pass 4  
Pass 5  
Pass 6  
Pass 7
```

每个 Pass 从原始图像中选择一部分像素，并形成自己的扫描线。

19.3 PNG Filter 在每个 Pass 内独立工作

对于 Filter：

```
Up = previous scanline in the same pass
```

而不是：

```
Up = previous row in the original image
```

这是实现 Adam7 Decoder 时最重要的状态管理之一。

19.4 Unfilter 后再映射回原始图像坐标

核心公式：

```
dst_x = x_start + pass_x × x_step  
dst_y = y_start + pass_y × y_step
```

因此 Adam7 的坐标恢复本身很简单。

真正增加实现复杂度的是：

```
Variable Scanline Length
      +
Pass-local Filter Dependency
      +
Packed Samples
      +
Scatter Frame-buffer Write
```

20. 总结

Adam7 可以概括为：

将 PNG 图像按照固定的二维采样模式拆分为 7 个由稀到密的 Pass，每个 Pass 独立形成扫描线并执行 PNG Filter，压缩后依次存入 PNG 数据流；解码时逐 Pass 解滤波并将像素映射回完整图像，从而实现渐进式图像重建。

对于软件 Decoder，Adam7 的核心是 Pass 管理与坐标映射。

对于硬件 Decoder，真正需要重点关注的则是：

- Pass 宽度动态变化；
- Filter previous-line buffer；
- 低 bit-depth sample packing；
- 非连续 Frame Buffer 地址；
- DDR / VDMA 的稀疏写入效率。

09. 颜色、Alpha 与输出

1. 各 Color Type 的输出

1.1 Grayscale

低位深灰度通常按满范围缩放到输出位深。例如 n-bit 到8-bit:

```
out = round(sample * 255 / (2^n - 1))
```

不要简单左移而导致最大值不能映射到255。

1.2 Truecolor

按 R、G、B 顺序读取。若输出 RGBA，Alpha 补最大值。

1.3 Indexed-color

Sample 是 PLTE Index，不是灰度。先查 RGB，再从 tRNS 查 Alpha；无对应 tRNS 条目的 Alpha 为255。

1.4 带 Alpha 类型

Type 4 为 G、A；Type 6 为 R、G、B、A。PNG Alpha 是 straight alpha。

2. 16-bit 到8-bit

最简单的高字节截断：

```
out8 = sample16 >> 8
```

速度快，但不是数学上最准确的满范围舍入。更精确：

```
out8 = round(sample16 * 255 / 65535)
```

如果下游支持16-bit，应优先保留原精度。sBIT 描述源数据中的有效精度，可帮助编辑器避免在反复转换中虚构精度。

3. Gamma、颜色和 Alpha

完整 Viewer 的逻辑概念上是：

1. 根据 iCCP、cICP、sRGB、gAMA/cHRM 等确定源颜色空间；
2. 把颜色通道转换到合适的线性或连接空间；

3. 在线性光意义下执行 Alpha 合成；
4. 转换到显示设备空间；
5. 量化到显示格式。

基础硬件 Decoder 常只恢复编码 Sample，把颜色管理交给后级。接口文档必须说明输出是“原始 PNG Sample”还是“已经转换的显示 RGB”，否则不同模块会对 Gamma 和 Alpha 作重复处理。

10. 编码器

1. 编码器总流程

```
source pixels
-> choose color type and bit depth
-> optional palette and transparency
-> optional Adam7 extraction
-> serialize each pass into scanlines
-> choose one filter type per scanline
-> DEFLATE all filtered scanlines as one stream
-> add zlib header and Adler-32
-> split bytes into consecutive IDAT chunks
-> add chunks and CRCs
```

IDAT 大小是封装选择，不应改变解压结果。编码器可以按便于流式输出或网络传输的尺寸切分。

2. Filter 选择

规范不规定最佳 Filter 选择算法。常见启发式：

1. 对当前行分别计算五种 Filter；
2. 把 Filtered byte 解释为有符号残差；
3. 计算绝对值和；
4. 选择得分最低者。

```
score = sum(abs((int8)filtered_byte))
```

它只近似预测 DEFLATE 效果。更高压缩率实现可以在候选行或候选组上实际试压缩，但计算成本更高。

一般经验：

- None：噪声、已经压缩式分布或低开销模式；
- Sub：水平渐变、宽色带；
- Up：垂直重复；
- Average：水平和垂直都相关；
- Paeth：二维平滑区域和边缘。

3. DEFLATE 编码优化

Decoder 是确定的，Encoder 的搜索策略却可复杂很多：

- Hash Table/Hash Chain 查找相同前缀；
- Greedy Match 立即选择当前最长匹配；
- Lazy Match 比较当前位置与下一位置；
- 限制 Search Depth 换速度；
- Block Splitting 在 Stored、Fixed、Dynamic 间估算成本；
- 统计频率并生成限制最大码长的 Huffman Tree；
- 根据 Header 成本判断 Dynamic 是否真正划算。

Compression Level 通常改变搜索深度、Lazy 策略和 Block 决策，不改变格式语义。

11. 硬件架构

1. 顶层模块

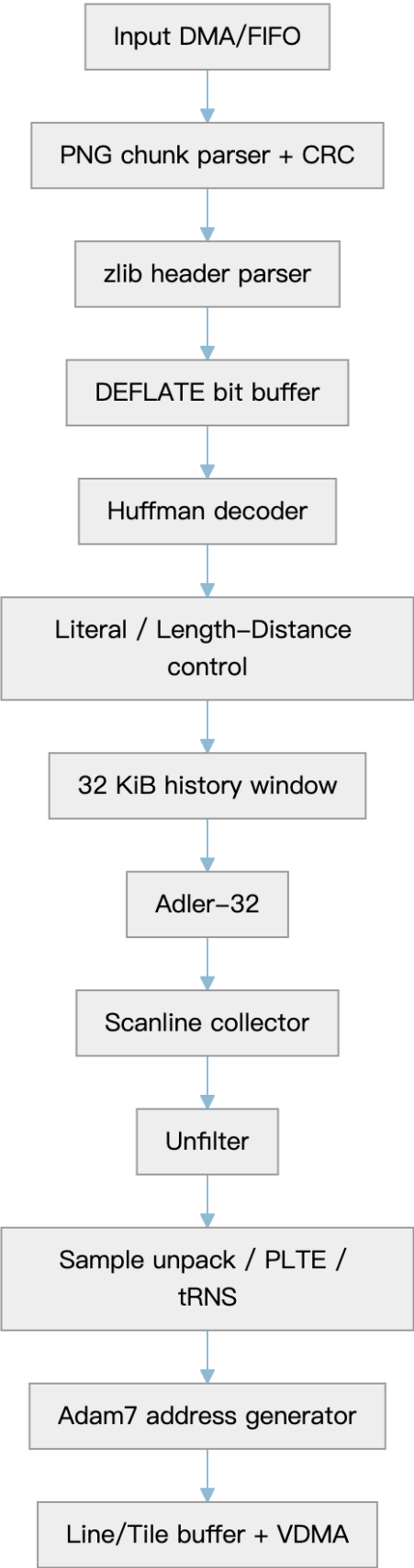


图 4

各层之间必须支持 valid/ready 或等价流控。下游行缓存或 VDMA 停顿会反压到 DEFLATE；输入端必须能暂停而不丢失 Bit Buffer 状态。

2. 与 JPEG Decoder 的复用

可以复用：

- Host/Register/DMA 框架；
- 输入输出 FIFO；
- 图像尺寸和 stride 管理；
- 像素格式转换中的部分通道逻辑；
- Frame Buffer、VDMA、中断和错误上报。

通常不能直接复用：

- JPEG Marker Parser 作为 PNG Chunk Parser；
- JPEG Huffman 语义作为 DEFLATE Huffman 语义；
- IDCT、反量化路径；
- MCU/Block 调度作为 PNG Scanline 调度；
- JPEG 行输出假设作为 Adam7 Scatter。

JPEG 以频域 Block/MCU 为核心；PNG 以连续 byte stream、LZ77 History 和 Scanline Predictor 为核心。

3. DEFLATE 硬件难点

3.1 位长可变

一次 Symbol 消耗的 bit 数变化，Length/Distance 后还有 Extra Bits。Bit Buffer 需要支持：

- 查看低 N bit；
- 一拍消费可变数量；
- 跨输入 word 补充；
- Byte Alignment；
- Block 类型切换。

3.2 Dynamic Tree 构建

必须先解 Code Length Tree，再产生两棵数据 Tree。构表阶段没有像素输出，造成启动延迟。

可以用：

- 小型状态机与 RAM；
- Canonical Code 生成单元；

- 一级表加二级表；
- Fixed 表 ROM 与 Dynamic 表 RAM 分离。

3.3 Match Copy 吞吐

distance 小于并行输出宽度时会发生同拍内递归依赖。例如 distance=1 表示重复同一 byte。多 byte/cycle 设计必须：

- 对短周期 Pattern 做展开；
- 或降低该 Match 的输出宽度；
- 或使用迭代旁路网络。

3.4 Window RAM

同时存在读源、写目的和可能的多 byte 访问。需考虑：

- Bank 划分；
- Wrap；
- Read-after-write；
- distance 很小时的 Forwarding；
- Block 边界不清空；
- zlib 流开始时有效 History 为0。

4. Scanline 硬件依赖

反滤波依赖图：

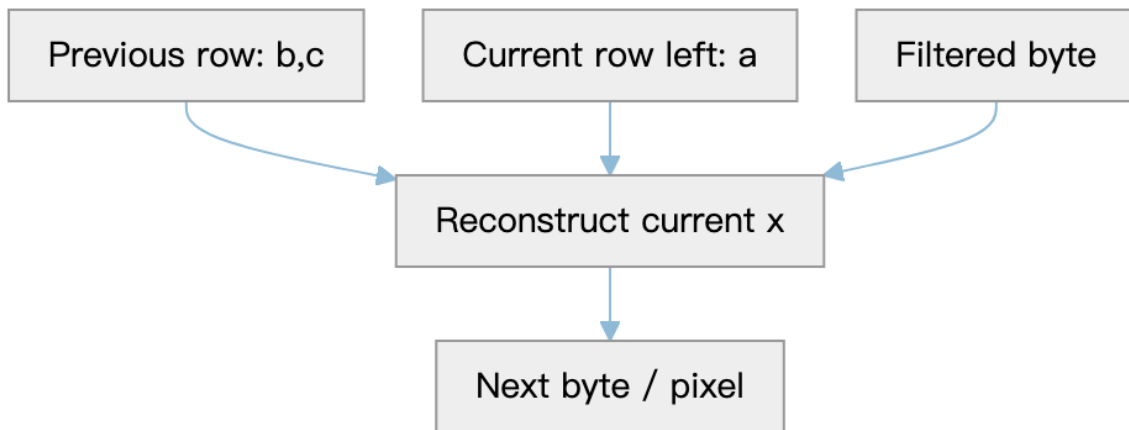


图 5

None、Up 容易按多 byte 并行；Sub、Average、Paeth 有行内递推。RGB/RGBA 的依赖距离是 bpp，可在通道对应 lane 之间形成若干链，但 byte 打包、低位深和总线对齐会使固定 lane 方案复杂化。

Paeth 的组合路径包括加减、绝对值和三级比较，可能成为高频设计关键路径。可通过：

- 预测值流水；
- 多 lane 错位；
- 小批次前缀展开；
- 降低单周期输出宽度；
- 根据 Filter Type 绕过不用的逻辑。

5. 能否像 WPP 一样多行推进

答案是“可以构造有限的 Wavefront，但不能直接照搬视频编码 WPP”。

限制来自两层：

1. DEFLATE 只按原始序列顺序吐出 byte，后续行数据尚未解压时不能提前重建；
2. Up/Average/Paeth 需要上一行数据，Sub/Average/Paeth 还需要本行左侧数据。

若先缓存多行 Filtered Data，可以让多个 Unfilter Worker 错位推进：

- Row r 的位置 x 依赖 Row $r-1$ 的 x 已完成；
- 对 Average/Paeth，还依赖 Row r 当前 x -bpp 已完成；
- 因而可形成沿 x 方向右移的 Wavefront。

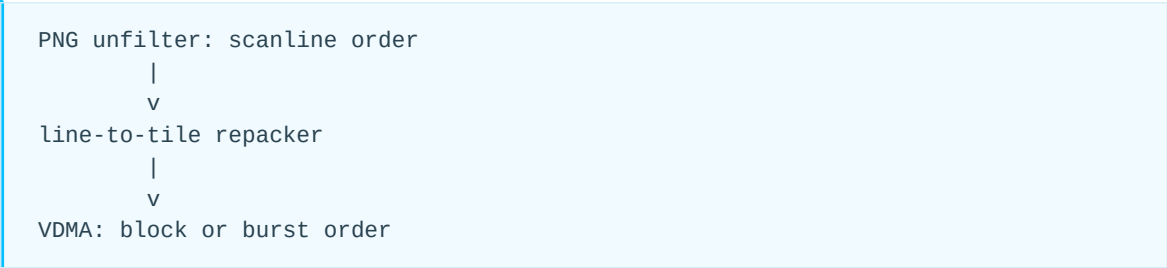
但实际收益取决于：

- DEFLATE 输出是否足以供给多个 Worker；
- 行宽；
- Filter Type 分布；
- SRAM 端口和带宽；
- 输出 VDMA 是否能接受错位多行；
- 为多行数据增加的缓存面积。

缓冲32行并不会自动获得32倍吞吐。若 DEFLATE 平均只产生1 byte/cycle，后级32行并行也无法突破供给瓶颈。

6. 行输出与块输出

PNG 天然按行重建，但系统 VDMA 可能要求 Tile/Block。建议在语义上分两层：



可选方案:

方案	缓存	优点	缺点
VDMA 支持逐行	1 – 2行	面积最小	需改 VDMA 接口或调度
缓存一个 Tile 高度	tile_h 行	块输出自然	面积随最大宽度增加
缓存完整图像	最大	最简单解耦	通常不可接受
多行环形 Buffer	N行	支持 Wavefront 和 Burst	控制、Bank 和顺序复杂

“32K 行缓存”与“32 KiB LZ77 Window”是完全不同的概念。前者可能极大，后者是 DEFLATE 的固定上限。

7. Buffer 估算

设最大宽度 W 、bits_per_pixel= B :

```
row_bytes = ceil(W*B/8)
two_row_filter_buffer = 2*row_bytes
N_row_buffer = N*row_bytes
```

例如 32768 像素宽、RGBA8:

```
row_bytes = 32768*4 = 131072 bytes
32 rows = 4 MiB
```

如果目标是片上 SRAM，这个规模必须与芯片面积预算对照。工程上往往更合理的是:

- 让 VDMA 接受行或较小 Strip;
- 用外部内存作重排;
- 限制硬件 Profile 的最大宽度;
- 使用少量行缓存配合块化写出。

8. 性能模型

端到端吞吐是最慢阶段的结果:

```
T = min(  
    input byte rate,  
    Huffman symbol rate converted to output bytes,  
    LZ77 copy rate,  
    unfilter byte rate,  
    pixel unpack rate,  
    VDMA acceptance rate  
)
```

不能只用 Pixels/Cycle 描述 DEFLATE，因为一个 Huffman Symbol 可能输出：

- 1 byte Literal;
- 3至258 byte Match;
- 0 byte End-of-block。

建议采集：

- Block 类型计数；
- Literal 和 Match 数；
- 平均 Match Length；
- distance 分布；
- Huffman 表构建周期；
- Window 冲突停顿；
- 各 Filter Type 行数；
- Unfilter 停顿；
- VDMA Backpressure 周期。

12. 错误、安全与验证

1. 分层错误模型

层	典型错误
PNG	Signature、Chunk 顺序、长度、CRC、未知 Critical
IHDR	非法尺寸、Color Type/Bit Depth、Method
zlib	Header mod31、CM、CINFO、FDICT、Adler
DEFLATE	BTYPE=11、Tree 非法、保留符号、越界 Distance、截断
Scanline	非法 Filter Type、输出过多/过少
Pixel	Palette Index 越界、缺失 PLTE
Adam7	Pass 长度不匹配、地址越界

错误上报应保留首个根因，不要让后续级联错误覆盖它。

2. 安全性

解析不可信 PNG 时必须防范：

- width*height*channels 的整数溢出；
- Chunk Length 诱导的超大分配；
- 极小压缩文件产生极大输出；
- 文本或 ICC Chunk 解压膨胀；
- 截断流导致无限等待；
- 恶意 Dynamic Tree 触发表越界；
- distance 超过有效 History；
- Adam7 坐标计算溢出；
- 过度 CPU 消耗和内存占用。

建议在解码前或解码过程中设置：

- 最大宽高、最大像素数；
- 最大 Chunk 长度；
- 最大元数据解压长度；
- 预期图像数据输出上限；
- 输入与输出超时；

- 所有计数器的饱和或溢出检查。

对静态 PNG，IHDR 已足以计算精确预期 Scanline 输出长度。DEFLATE 输出超过该值应立即报错；结束时不足也应报错。

3. 验证矩阵

3.1 格式维度

- 五种 Color Type;
- 所有合法 Bit Depth;
- 单 IDAT、多 IDAT、极小 IDAT;
- Ancillary Chunk 前后位置;
- CRC 正确与错误;
- Adam7 0/1。

3.2 DEFLATE 维度

- Stored、Fixed、Dynamic;
- 多 Block;
- 每种 Block 作为最后或非最后;
- 跨 Block LZ77 引用;
- distance=1;
- length=258;
- Window Wrap;
- Dynamic 重复符号16、17、18;
- 不完整、oversubscribed、保留符号和截断流。

3.3 Filter 维度

- 五种 Filter;
- 第一行;
- 行首前 bpp 个 byte;
- width=1;
- 低位深;
- 16-bit;
- Filter Type 每行变化;
- 每个 Adam7 Pass 的第一行。

3.4 参考比对

建议建立：

```
PNG test file
-> reference software decoder
-> RTL/C model under test
-> byte-exact compare at several checkpoints
```

检查点包括：

1. IDAT 聚合字节；
2. DEFLATE 输出；
3. 每行 Unfilter 输出；
4. 解包 Sample；
5. 最终 Pixel；
6. CRC、Adler 和错误码。

分层比对能快速判断错误属于 Bitstream、LZ77、Filter 还是像素解释。

13. 贯通实例

1. 最小文件结构

一张最小静态 PNG 至少包含：

Signature
IHDR
IDAT
IEND

逐层解析方法：

- 1. 识别 Signature；
- 2. 读取 IHDR，得到尺寸和像素布局；
- 3. 计算 row_bytes、bpp 和预期解压长度；
- 4. 对 IDAT 计算 Chunk CRC，同时把 Data 送入 zlib；
- 5. 检查 CMF/FLG；
- 6. 解 DEFLATE Block；
- 7. 对输出更新 Adler；
- 8. 以 Filter Type+row_bytes 切行；
- 9. Unfilter；
- 10. 解包像素；
- 11. 比较 zlib 尾部 Adler；
- 12. 验证 IEND。

2. 一个 Filter 算例

假设 RGB8，宽2，上一行和当前 Filtered Data 为：

previous = [10, 20, 30, 40, 50, 60]
filter = Paeth
filtered = [2, 1,255, 5, 3, 2]
bpp = 3

前 bpp 个 byte 没有左邻居；本例 bpp=3，因此是前三个 byte：

- i=0: a=0,b=10,c=0, Paeth 选 b=10, x=2+10=12;
- i=1: a=0,b=20,c=0, x=1+20=21;
- i=2: a=0,b=30,c=0, x=255+30 mod256=29。

从 $i=\text{bpp}$ 开始， a 是当前行 $i\text{-bpp}$ 位置的已恢复值， b 是上一行同位置， c 是上一行 $i\text{-bpp}$ 位置；本例可直观理解为“向前3个 byte”。必须使用刚恢复的 `current`，而不是 `Filtered byte` 作为 a 。

3. 完整实现骨架

```
decode_png(input):
    require(read(8) == PNG_SIGNATURE)

    state = init_png_state()
    z = init_zlib_state()

    while not state.seen_iend:
        length = read_u32_be()
        type = read(4)
        validate_chunk_header(state, type, length)

        crc = crc32_init()
        crc.update(type)

        repeat length:
            d = read_byte()
            crc.update(d)

            if type == IDAT:
                z.push(d)
            else:
                consume_chunk_data(state, type, d)

        require(crc.finish() == read_u32_be())
        finish_chunk(state, type)

    require(z.finished)
    require(state.image_data_exact_length)
    return state.output
```

真实实现通常让 `z.push` 产生零个或多个未压缩 byte；这些 byte 立即进入 `Scanline Reconstructor`。

14. PNG 图像测试集与验证资源汇总

1. 概述

PNG (Portable Network Graphics) 生态中存在多套用于验证 PNG 编解码器正确性的测试资源，覆盖：

- PNG 基础格式验证
- 不同颜色类型与位深
- Filter 算法验证
- Adam7 隔行扫描验证
- Chunk 解析验证
- zlib/DEFLATE 压缩组合验证
- 异常 PNG 与鲁棒性测试

目前没有一个单独覆盖全部场景的“官方完整 PNG 测试集”。实际开发中通常组合使用：

1. PngSuite (PNG 生态广泛使用的经典测试集)
2. libpng 官方测试数据
3. OSS-Fuzz PNG Corpus
4. 浏览器和大型软件 PNG 回归测试集合

2. PngSuite

2.1 简介

PngSuite 是 PNG 领域最经典、使用最广泛的测试图片集合之一，由 Willem van Schaik 创建。其网站自称带引号的“official” test-suite，但它不是由 W3C 或 ISO 维护的正式一致性测试套件。

它的目标不是提供普通图片素材，而是验证 PNG 解码器是否正确支持 PNG 标准定义的各种特性。

来源：

- 官网：<http://www.schaik.com/pngsuite/>
- 下载：<https://github.com/lunapaint/pngsuite>

2.2 测试内容

PngSuite 覆盖：

分类	测试内容
PNG 基础结构	PNG signature、IHDR、IDAT、IEND
颜色类型	Grayscale、Truecolor、Indexed-color、Grayscale with Alpha、Truecolor with Alpha
Alpha	Transparency、Alpha channel
位深	1/2/4/8/16 bit
Filter	None、Sub、Up、Average、Paeth
Interlace	Adam7
Chunk	Ancillary chunks
Gamma	gAMA
Color profile	sRGB、cHRM
错误情况	非标准或异常 PNG

2.3 文件命名规则

PngSuite 文件名包含编码信息，通常依次表示测试特征、测试参数、是否交错、颜色类型编码和位深。

例如：

```
basn6a08.png
```

可以拆为：

字段	含义
bas	basic 图像测试前缀
n	non-interlaced；对应的 i 表示 interlaced
6a	Color Type 6，Truecolor with Alpha
08	8-bit depth

颜色类型编码是完整的 0g、2c、3p、4a 或 6a，其中数字与描述字符不能拆成两个独立字段。交错标记为 n（non-interlaced）或 i（interlaced）。

本仓库所含 PngSuite 样本的完整分类、文件名与验证重点见 14-1. PngSuite 测试集分类与图片说明。

常见示例：

文件	说明
basn0g01.png	灰度 1bit
basn0g08.png	灰度 8bit
basn0g16.png	灰度 16bit
basn2c08.png	RGB 8bit
basn3p08.png	Palette 8bit
basn4a08.png	Gray + Alpha
basn6a08.png	RGBA

3. libpng 官方测试资源

3.1 简介

libpng 是 PNG 最主要的参考实现之一。

其测试目录包含大量用于验证 PNG 编解码正确性的测试程序和测试数据。

来源：

GitHub：

<https://github.com/pnggroup/libpng>

3.2 主要测试内容

目录：

contrib/pngsuite
contrib/testpngs

主要包括：

pngtest

用途：

验证 PNG 文件：

```
PNG file
↓
decode
↓
encode
↓
compare
```

用于检查：

- 解码正确性
- 编码一致性
- 数据完整性

pngvalid

用途：

自动生成并验证大量 PNG 类型。

覆盖：

- 不同 bit depth
- 不同 color type
- 不同 filter
- Adam7 interlace
- gamma
- chunk组合

3.3 下载

GitHub：

<https://github.com/pnggroup/libpng>

下载：

```
git clone https://github.com/pnggroup/libpng.git
```

测试资源位置：

```
libpng/
├── tests/           # 测试脚本
└── contrib/
    ├── pngsuite/   # PngSuite 测试图片
    └── testpngs/   # libpng 补充测试图片
```

4. PNG Specification 示例资源

4.1 简介

PNG 标准文档中包含部分示例和规范说明。

标准：

ISO/IEC 15948 Portable Network Graphics (PNG)

相关资料：

PNG Specification：

<https://www.w3.org/TR/png/>

4.2 用途

主要用于：

- 标准行为确认
- Chunk 定义参考
- 编码规则验证

特点：

- 权威性高
- 示例数量有限

5. OSS-Fuzz PNG Corpus

5.1 简介

OSS-Fuzz 是 Google 发起的大规模开源软件模糊测试项目。

其中包含大量 PNG 相关测试样本。

来源：

<https://github.com/google/oss-fuzz>

5.2 测试特点

相比标准测试集，OSS-Fuzz 更关注：

- 异常 PNG
- 崩溃触发样本
- 边界条件
- 安全漏洞

覆盖：

类型	示例
文件损坏	truncated PNG
Chunk异常	invalid chunk
CRC错误	bad CRC
压缩异常	invalid zlib
极端尺寸	huge image
解码异常	parser corner case

5.3 下载

OSS-Fuzz 项目配置：

<https://github.com/google/oss-fuzz>

仓库中的 `projects/libpng`、`projects/libpng-proto` 等目录只包含构建和 fuzz target 配置，不直接存放可下载的 corpus。持续演化的 corpus 应按 OSS-Fuzz 的 corpus 下载说明获取：

<https://google.github.io/oss-fuzz/advanced-topics/corpora/>

已公开问题的最小崩溃样本通常作为 reproducer 附在具体 issue 中，复现流程见：

<https://google.github.io/oss-fuzz/advanced-topics/reproducing/>

6. 浏览器与大型软件 PNG 回归测试

6.1 Mozilla PNG Tests

源码入口：

<https://searchfox.org/mozilla-central/source/image/test>

用途：

浏览器 PNG 解码测试。

覆盖：

- 浏览器兼容性
- 异常 PNG
- 历史 bug 回归

6.2 Chromium PNG Regression Tests

源码入口：

https://source.chromium.org/chromium/chromium/src/+main:third_party/blink/web_tests/images/

用途：

验证浏览器环境中的 PNG 解码稳定性。

覆盖：

- 历史 bug
- 特殊 PNG
- 安全问题

浏览器回归样本通常分散在图像解码器、布局测试和具体 bug 的测试目录中，不是单独发布的完整 PNG 测试包。使用时应记录所选样本的仓库版本和原始路径，以便结果可复现。

7. 推荐下载组合

标准兼容性测试

推荐：

```
PngSuite
+
libpng test
```

覆盖：

- PNG 标准功能
- 正常图片解码

鲁棒性测试

推荐：

```
OSS-Fuzz PNG corpus
+
浏览器 regression samples
```

覆盖：

- 异常输入
- 边界情况
- 安全测试

8. 测试资源对比

测试集	权威性	正常 PNG	异常 PNG	标准覆盖	下载
PngSuite	★★★	✓✓✓	✓	★★★★	http://www.schaik.com/pngsuite/
libpng test	★★★	✓✓✓	✓✓	★★★★	https://github.com/pnggroup/libpng
PNG Spec examples	★★★	✓	-	★★★★	https://www.w3.org/TR/png/
OSS-Fuzz	★★★	✓	✓✓✓	★★★★	https://github.com/google/oss-fuzz
Browser regression	★★★	✓	✓✓✓	★★★★	Chromium/Mozilla repositories

9. 总结

PNG 测试资源体系可以分为：

PNG 标准验证
-- PngSuite
-- libpng test
异常与安全验证
-- OSS-Fuzz
-- Browser regression tests

其中：

- PngSuite 是 PNG 标准兼容性的核心测试集。
- libpng test 是工程实现验证的重要参考。
- OSS-Fuzz 适合测试异常输入和鲁棒性。
- 浏览器测试集 提供大量真实世界历史问题样本。

实际开发 PNG 软件时，通常组合使用以上资源，而不是依赖单一测试集。

14-1. PngSuite 测试集分类与图片说明

1. 使用范围

PngSuite 是 Willem van Schaik 创建的经典 PNG 测试图片集合，用于验证 Viewer、Converter、Editor 和 Decoder 对 PNG 格式特性的支持。它不是 W3C 或 ISO 维护的正式一致性测试套件，但长期被 PNG 实现广泛采用。

本仓库保存了 PngSuite-2017jul19 的原始 PNG，以及便于自动比对的 BMP 和 JSON 参考数据：

```
pngsuite/  
├── png/      原始 PNG 测试图片  
├── bmp/      转换后的 BMP 参考图片  
└── json/     RGBA 数组；部分 16-bit 图片另有 _t08 版本
```

上游来源：

- PngSuite 官网：<http://www.schaik.com/pngsuite/>
- 本仓库采用的 GitHub 镜像：<https://github.com/lunapaint/pngsuite>

PngSuite 主要覆盖：

- 五种 Color Type 与全部合法 Bit Depth；
- Adam7、奇数尺寸和极小尺寸；
- bKGD、tRNS、gAMA、sBIT、pHYs、文本等 Ancillary Chunk；
- 五种 Scanline Filter；
- 多 IDAT 与不同 zlib 压缩级别；
- Signature、IHDR、CRC 等损坏输入。

它不能替代完整的规范一致性测试、资源上限测试和持续 Fuzzing。实际项目仍应结合第 14 章介绍的 libpng、OSS-Fuzz 和真实回归样本。

2. 文件命名规则

多数文件名采用以下结构：

```
测试特征与参数 | n/i | 颜色类型编码 | 位深 | .png
```

例如 g04n2c08.png：

字段	含义
g04	Gamma 测试，文件 Gamma 约为 0.45
n	non-interlaced; i 表示 Adam7 interlaced
2c	Color Type 2, Truecolor
08	8-bit depth

颜色类型编码是不可拆分的整体：

编码	Color Type	含义
0g	0	Grayscale
2c	2	Truecolor
3p	3	Indexed-color
4a	4	Grayscale with Alpha
6a	6	Truecolor with Alpha

basn6a08.png 因此应读作 bas + n + 6a + 08：basic、非交错、Truecolor with Alpha、8-bit。不能把 6a 拆成相互独立的 6 与 a 字段。

3. Basic Image：颜色类型与位深

basn* 是非交错基础图片，basi* 是内容对应的 Adam7 版本。两组均覆盖所有合法的 Color Type/Bit Depth 组合。

文件模式	Color Type	Bit Depth	主要检查点
bas?0g01/02/04/08/16.png	0	1、2、4、8、16	灰度 Sample、低位深打包、16-bit 大端顺序
bas?2c08/16.png	2	8、16	RGB 通道顺序与精度
bas?3p01/02/04/08.png	3	1、2、4、8	PLTE、Palette Index 与低位深打包
bas?4a08/16.png	4	8、16	灰度与 Alpha 通道
bas?6a08/16.png	6	8、16	RGBA 通道顺序与精度

这里的 ? 为 n 或 i。对 Indexed-color 图片，PLTE 是必需 Chunk；其他类型是否带 Ancillary Chunk 应以实际文件为准，不应仅凭文件名假定完整 Chunk 列表。

4. Adam7 与尺寸边界

4.1 Adam7

basi* 与所有文件名中交错位为 i 的样本用于检查：

- 七个 Pass 的尺寸和顺序；
- 每个非空 Pass 独立重置 Previous Row；
- Pass 内 Scanline Filter；
- Sample 回填到原图坐标；
- 小图导致部分 Pass 为空的情况。

4.2 奇数与极小尺寸

sNn3p*.png 和 sNNi3p*.png 成对提供非交错与 Adam7 版本：

文件范围	测试重点
s01* 至 s09*	1 到 9 像素附近的极小尺寸，覆盖空 Pass 和字节尾部 Padding
s32* 至 s40*	32 到 40 像素附近的尺寸，覆盖跨字节与 Pass 边界变化

这组样本特别适合检查 Adam7 的 pass_width、pass_height 和预期解压长度计算。

5. Transparency 与 Background

5.1 tRNS

文件模式	测试重点
tp0n0g08.png、tp0n2c08.png、tp0n3p08.png	无透明的参考图片
tp1n3p08.png	Indexed-color 透明且无 bKGD
tm3n3p02.png	Palette 中多个透明度级别
tbb*、tbg*、tbr*、tbw*、tby*	tRNS 与黑、灰、红、白、黄等背景组合

对于 Color Type 0/2，tRNS 表示一个完全透明的灰度值或 RGB 值；对于 Color Type 3，它是与 PLTE 对应的 Alpha 表。不要把 tbbn2c16.png 的 tRNS 当作完整 Alpha 通道。

5.2 bKGD 与 Alpha

文件	测试重点
bgai4a08.png、bgai4a16.png	交错灰度 Alpha，无指定背景
bgan6a08.png、bgan6a16.png	非交错 RGBA，无指定背景
bgbn4a08.png	灰度 Alpha 与黑色背景

文件	测试重点
bggn4a16.png	灰度 Alpha 与灰色背景
bgwn6a08.png	RGBA 与白色背景
bgyn6a16.png	RGBA 与黄色背景

bKGD 只是建议的显示背景，不是图像自身 Alpha，也不会把 PNG 数据改成预合成颜色。Decoder 可输出原始 Sample；Viewer 决定是否采用 bKGD 合成。

6. Gamma 与颜色相关 Chunk

6.1 gAMA

g03*、g04*、g05*、g07*、g10*、g25* 分别提供不同文件 Gamma。每个值包含 Grayscale 16-bit、Truecolor 8-bit 和 Indexed-color 4-bit 版本。

这组图片应在正确颜色管理后呈现相近外观。仅比较解码后的原始 Sample 时，各文件不一定逐字节相同。

6.2 cHRM、sBIT、hIST 与建议调色板

文件模式	Chunk/特性	测试重点
ccw*	cHRM	主色与白点
cs3*、cs5*、cs8*	sBIT	存储位深中的有效位数
ch1*、ch2*	hIST	Palette Entry 使用频率
pp0*	PLTE	Truecolor/RGBA 中的建议调色板
ps1*、ps2*	sPLT	Suggested Palette

sBIT 文件使用 cs 前缀；s01* 等文件属于尺寸测试，两者不能混为一类。

7. Scanline Filter

文件模式	Filter Type	测试重点
f00n0g08.png、f00n2c08.png	0	None
f01n0g08.png、f01n2c08.png	1	Sub
f02n0g08.png、f02n2c08.png	2	Up
f03n0g08.png、f03n2c08.png	3	Average
f04n0g08.png、f04n2c08.png	4	Paeth

文件模式	Filter Type	测试重点
f99n0g04.png	多种	每行自适应选择不同 Filter Type

Filter Type Byte 位于每个非空 Scanline 开头。Filter 按序列化后的 byte 工作，不能按抽象像素或通道直接套公式。

8. Ancillary Chunk

8.1 文本与 Exif

文件	测试重点
ct0n0g04.png	无文本的参考图片
ct1n0g04.png	tEXt 文本
ctzn0g04.png	zTXt 压缩文本
cten0g04.png	iTXt 英文文本
ctfn0g04.png	iTXt 芬兰语文本
ctgn0g04.png	iTXt 希腊语文本
cthn0g04.png	iTXt 印地语文本
ctjn0g04.png	iTXt 日语文本
exif2c08.png	eXIf Profile

8.2 时间与物理尺寸

文件模式	Chunk	测试重点
cm0n0g04.png、cm7n0g04.png、cm9n0g04.png	tIME	不同日期的修改时间
cdfn2c08.png	pHYs	8×32 的扁平像素
cdhn2c08.png	pHYs	32×8 的高像素
cdsn2c08.png	pHYs	方形像素
cdun2c08.png	pHYs	带 metre 单位的物理尺寸

pHYs 在单位未知时只表达像素宽高比；单位为 metre 时才能换算物理尺寸。它不直接存储 DPI。

9. IDAT 与 zlib

9.1 多 IDAT

文件模式	测试重点
oi1n0g16.png、oi1n2c16.png	一个 IDAT
oi2n0g16.png、oi2n2c16.png	两个 IDAT
oi4n0g16.png、oi4n2c16.png	四个大小不同的 IDAT
oi9n0g16.png、oi9n2c16.png	大量长度为 1 byte 的 IDAT

所有 IDAT Data 必须按顺序拼成同一个 zlib Datastream；Chunk 边界与 zlib Header、DEFLATE Block、Scanline 均无固定对齐关系。

9.2 zlib 压缩级别

文件	zlib Level
z00n2c08.png	0
z03n2c08.png	3
z06n2c08.png	6
z09n2c08.png	9

这些文件解码到相同图像，但压缩器可能选择不同的 Stored、Fixed、Dynamic Block 和 LZ77/Huffman 表示。Decoder 不应依赖特定压缩级别产生的位流形态。

10. 损坏输入

x* 文件是预期解码失败的负向测试：

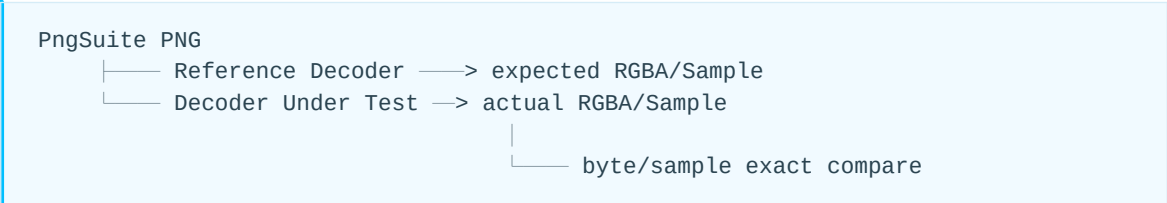
文件	预期错误
xs1n0g01.png、xs2n0g01.png、xs4n0g01.png、xs7n0g01.png	PNG Signature 损坏
xcrn0g04.png、xlfn0g04.png	传输过程中插入 CR/LF
xhdn0g08.png	IHDR CRC 错误
xcsn0g01.png	IDAT CRC 错误

文件	预期错误
xc1n0g08.png、xc9n2c08.png	非法 Color Type
xd0n2c08.png、xd3n2c08.png、xd9n2c08.png	非法 Bit Depth
xdtn0g01.png	缺少 IDAT

测试负向样本时应断言“以正确错误类别失败”，而不只是“不崩溃”。生产 Decoder 还必须补充截断、超大尺寸、解压膨胀、非法 Huffman Tree、越界 Distance 和输出超限等 PngSuite 未完全覆盖的安全测试。

11. 推荐验证方法

建议建立分层、自动化的验证流程：



- 1. 正向样本与可信参考 Decoder 或仓库 JSON 逐 Sample 比对；
 - 2. 16-bit 到 8-bit 的比较必须固定舍入规则，避免把量化差异误判为解码错误；
 - 3. 在 IDAT 聚合、DEFLATE 输出、Unfilter、Sample 解包和最终像素处设置检查点；
 - 4. 对颜色管理样本区分“原始 Sample 一致”和“显示外观一致”；
 - 5. 对 x* 样本检查错误分类、资源释放和无越界访问；
 - 6. 将 PngSuite 与 libpng 测试、OSS-Fuzz Corpus 及项目历史回归样本组合使用。
- PngSuite 最适合回答“常见 PNG 特性是否正确实现”；它不应被当作安全性、性能或全部现代 PNG Third Edition 特性的唯一验收标准。

15. 附录与结语

1. 附录 A 速查公式

```
bits_per_pixel = channels * bit_depth
row_bytes = ceil(width * bits_per_pixel / 8)
bpp = max(1, ceil(bits_per_pixel / 8))

non_interlaced_uncompressed_size =
    height * (1 + row_bytes)

pass_width =
    width <= x_start ? 0 :
    ceil((width - x_start) / x_step)

pass_height =
    height <= y_start ? 0 :
    ceil((height - y_start) / y_step)
```

2. 附录 B 常见误解

1. “每个 IDAT 都是一个 zlib 流。” 错。所有连续 IDAT Data 拼成一个 zlib 流。
2. “IDAT Data 就是 Raw DEFLATE。” 错。PNG Compression Method 0 使用带 CMF/FLG 和 Adler-32 的 zlib 包装。
3. “DEFLATE 解压后就是 RGB。” 错。先得到 Filter Type 与 Filtered Scanline。
4. “Filter 按像素预测。” 不精确。规范算法按序列化 byte 处理，左邻距离为 bpp。
5. “DEFLATE Block 在每行结束。” 错。Block 与 Scanline 无固定对齐关系。
6. “Block 结束要清空 32 KiB Window。” 错。LZ77 可跨 Block 引用。
7. “有 32 行缓存就能 32 行并行。” 错。还受 DEFLATE 串行输出、Filter 依赖、存储端口和 VDMA 限制。
8. “PNG 的 32K 指行缓存。” 错。通常指 DEFLATE 最大 32768-byte History Window。
9. “Alpha 也做 Gamma Correction。” 错。Alpha 是线性覆盖率。
10. “16-bit PNG 是小端。” 错。PNG Sample 和 Chunk 整数按高字节在前；Stored Block 的 LEN/NLEN 是 DEFLATE 内部例外，按 little-endian。

3. 附录 C APNG 概览

较新 PNG 规范把 APNG 纳入主规范。三个主要 Chunk：

- acTL：总帧数和循环次数；
- fcTL：帧区域、延时、Dispose 和 Blend；

- fdAT：非默认帧的压缩数据，带 Sequence Number。

静态图像仍由普通 IDAT 表示。APNG 的每个帧数据序列使用独立 zlib 数据流，不能把不同帧的压缩数据串成静态图像的同一 zlib 流。兼容静态 PNG 的 Decoder 可以忽略动画 Ancillary Chunk，并显示静态图像。

APNG 详细帧合成属于独立扩展主题；静态 PNG 的 Chunk、zlib、DEFLATE、Filter 和像素恢复知识仍是其基础。

4. 附录 D PNG 规范版本历史与演进

PNG 的核心数据模型在 1995 年冻结，此后的规范演进以向后兼容为原则：保留 Signature、四类 Critical Chunk、zlib/DEFLATE、五种 Filter 和 Adam7 等基础机制，主要通过新增 Ancillary Chunk、澄清行为和强化一致性要求扩展能力。

版本	年代与发布状态	主要功能	相对前代的新增与变化	实现与兼容性意义
PNG 1.0	1996-10-01, W3C Recommendation; 1997 年以 RFC 2083 发布	建立静态 PNG 的完整格式：灰度、真彩色、索引色，1 ~ 16-bit Sample, tRNS 与完整 Alpha, Chunk/CRC, zlib/DEFLATE, 逐行 Filter, Adam7, Gamma 与色度信息	首次标准化 IHDR、PLTE、IDAT、IEND 及 10 类 Ancillary Chunk; 提供无专利负担的 GIF 替代方案，并支持更高色深、完整 Alpha 和更强错误检测	今天的基础 Decoder 数据通路基本由此确定；规范当时只定义单幅静态图像
PNG 1.1	1998-10 批准、1998-12 发布；PNG Development Group 版本，未由标准组织批准	保持 1.0 位流和核心解码流程不变，增强颜色管理与建议调色板表达	新增 iCCP、sRGB、sPLT; 把 gAMA 重新定义为相对于期望显示输出；把 31-bit 上限扩展到所有四字节无符号整数，并澄清较小 zlib Window 合法	旧 Decoder 可按 Ancillary Chunk 规则忽略新增 Chunk；新实现应采用修订后的 gAMA 语义
PNG 1.2	1999-02 批准、1999-07 发布；PNG Development Group 版本，未由标准组织批准	在 1.1 基础上补足国际化文本能力	新增 iTXt, 用 UTF-8 表示任意语言文本，并支持语言标签、翻译后的 Keyword 和可选压缩；重排 Ancillary Chunk 的说明结构	为 Unicode 元数据提供标准路径；不认识 iTXt 的旧 Decoder 仍可安全忽略它

版本	年代与发布状态	主要功能	相对前代的新增与变化	实现与兼容性意义
PN G Se con d Ed ition	2003-11-10 ， W3C Re commenda tion； 对 应 ISO/IE C 15948	将 1.0、1.1、1.2 整理为正式、系统化的静态 PNG 功能规范，并定义 Datastream、Encoder、Decoder 和 Editor 的一致性条件	纳入 iCCP、iTXt、sRGB、sPLT 及已知勘误；引入 Concepts 与 Conformance 章节；强化部分 Encoder、Decoder、Editor 要求，但避免使既有合规文件失效	长期以来最常被传统库和硬件实现引用；实现静态 PNG 核心功能时仍是重要兼容基线
PN G T hird Edit ion	2022 年启 动修订； 2 025-06-24 成为 W3C Recommen dation	在静态 PNG 基础上正式覆盖动画、Exif、现代颜色空间、宽色域和 HDR 元数据	纳入 APNG 的 acTL、fcTL、fdAT，纳入 eXIf；新增 cICP、mDCV、cLLI；明确颜色 Chunk 优先级、image/apng 媒体类型、越界调色板索引与错误恢复，并合并 Second Edition 勘误	本书采用的现行 W3C 基准；基础静态解码路径保持兼容，但完整 Third Edition 实现还需处理动画和新增元数据
PN G F ourt h E ditio n	当前为 W 3C Editor's Draft； 草 案页面于 2026-07-28 更新	延续 Third Edition，并面向超宽色域、HDR 合成和内容来源验证完善元数据	草案新增 hRWL（HDR Reference White Level）和 caBX（C2PA Content Credentials）；允许 cHRM 用有符号整数表达负色度坐标；进一步澄清 16-bit Filter 与现代颜色处理	仅用于跟踪未来方向；在成为 W3C Recommendation 或 ISO 标准前，不应把草案要求当作现行强制要求，字段定义仍可能变化

PNG 文件的 Signature 和 IHDR 中都没有“规范版本号”。兼容性由 Chunk 机制逐项表达：未知 Ancillary Chunk 通常可以忽略，未知 Critical Chunk 则表示 Decoder 无法可靠解释图像。因此，“符合哪一版规范”是对文件所用功能和实现行为的描述，而不是文件头中的单一版本字段。

RFC 1950 和 RFC 1951 不属于 PNG 规范的代际版本：前者定义 zlib Wrapper 与 Adler-32，后者定义 DEFLATE 位流、Block、Huffman 和 LZ77；各代 PNG 规范通过 Compression Method 0 引用它们。

5. 附录 E 参考资料

1. W3C, PNG (Portable Network Graphics) Specification, Version 1.0:
<https://www.w3.org/TR/REC-png-961001>

2. PNG Development Group, PNG Specification Version 1.1 Revision History:
<http://www.libpng.org/pub/png/spec/1.1/PNG-History.html>

3. PNG Development Group, PNG Specification Version 1.2 Revision History:
<http://www.libpng.org/pub/png/spec/1.2/PNG-History.html>

4. W3C, Portable Network Graphics (PNG) Specification, Second Edition:
<https://www.w3.org/TR/2003/REC-PNG-20031110/>

5. W3C, Portable Network Graphics (PNG) Specification, Third Edition:
<https://www.w3.org/TR/png-3/>
6. W3C, Portable Network Graphics (PNG) Specification, Fourth Edition Editor's Draft:
<https://w3c.github.io/png/>
7. ISO/IEC 15948:2004, Information technology — Computer graphics and image processing —
Portable Network Graphics (PNG): Functional specification.
8. RFC 1950, ZLIB Compressed Data Format Specification version 3.3:
<https://www.rfc-editor.org/rfc/rfc1950>
9. RFC 1951, DEFLATE Compressed Data Format Specification version 1.3:
<https://www.rfc-editor.org/rfc/rfc1951>
10. W3C PNG Test Suite and implementation resources, 见 PNG Specification 的 Online resources。

6. 结语

PNG Decoder 的本质不是一个单独算法，而是一条必须严格分层的可逆数据链：

```
Chunk framing  
-> zlib wrapper  
-> DEFLATE entropy and dictionary decoding  
-> scanline prediction reversal  
-> sample and color interpretation
```

对软件实现，分层能降低错误定位成本；对硬件实现，分层能明确状态、缓冲、吞吐和反压边界。尤其应避免把 IDAT、DEFLATE Block、Scanline 和输出 Block 四种边界混为一谈。只要先建立正确的层次模型，再逐层验证，PNG 从格式到算法就会变成一套清晰、可实现、可测试的系统。